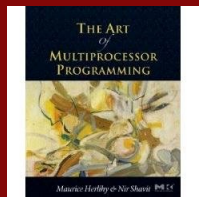
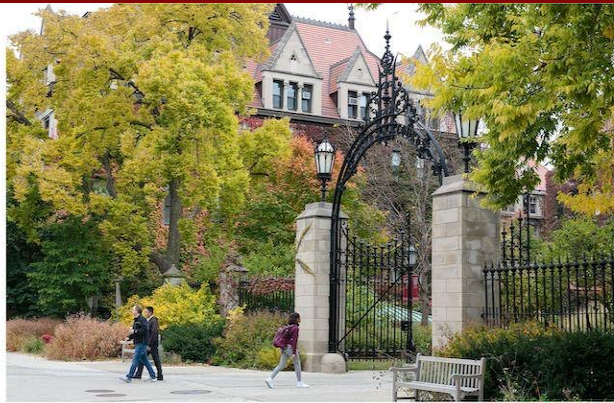


MPCS 52060 - Parallel Programming

M3: Principles of Mutual Exclusion



Original slides from “The Art of Multiprocessor Programming by Maurice Herlihy & Nir Shavit” With modifications by Lamont Samuels and Jan Hückelheim

Outline

- Previously:
 - Threads and processes
 - Fork and join, spawn threads, waitgroups
 - Atomic updates, e.g. `addInt()`
 - Caches
- This week:
 - Critical sections and locks
 - Properties of locks
 - Compare & Swap, a more powerful atomic operation
 - Different types of locks
 - Amdahl's law and scalability in more detail

Parallel Performance

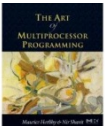
Speedup

$$\text{Speedup} = \frac{\text{1-thread execution time}}{\text{n-thread execution time}}$$

- Important metric: **speedup**
 - Ratio between time to solve a task with one processing unit versus solving the same problem with multiple processing units in parallel.
 - **Linear speedup** - algorithm runs P times faster on P processors. Rare in practice
 - **Superlinear speedup** - algorithm runs more than P times faster on P processors. Extremely rare in practice

Amdahl's Law

$$\text{Speedup} = \frac{1}{1 - p + \frac{p}{n}}$$



Amdahl's Law

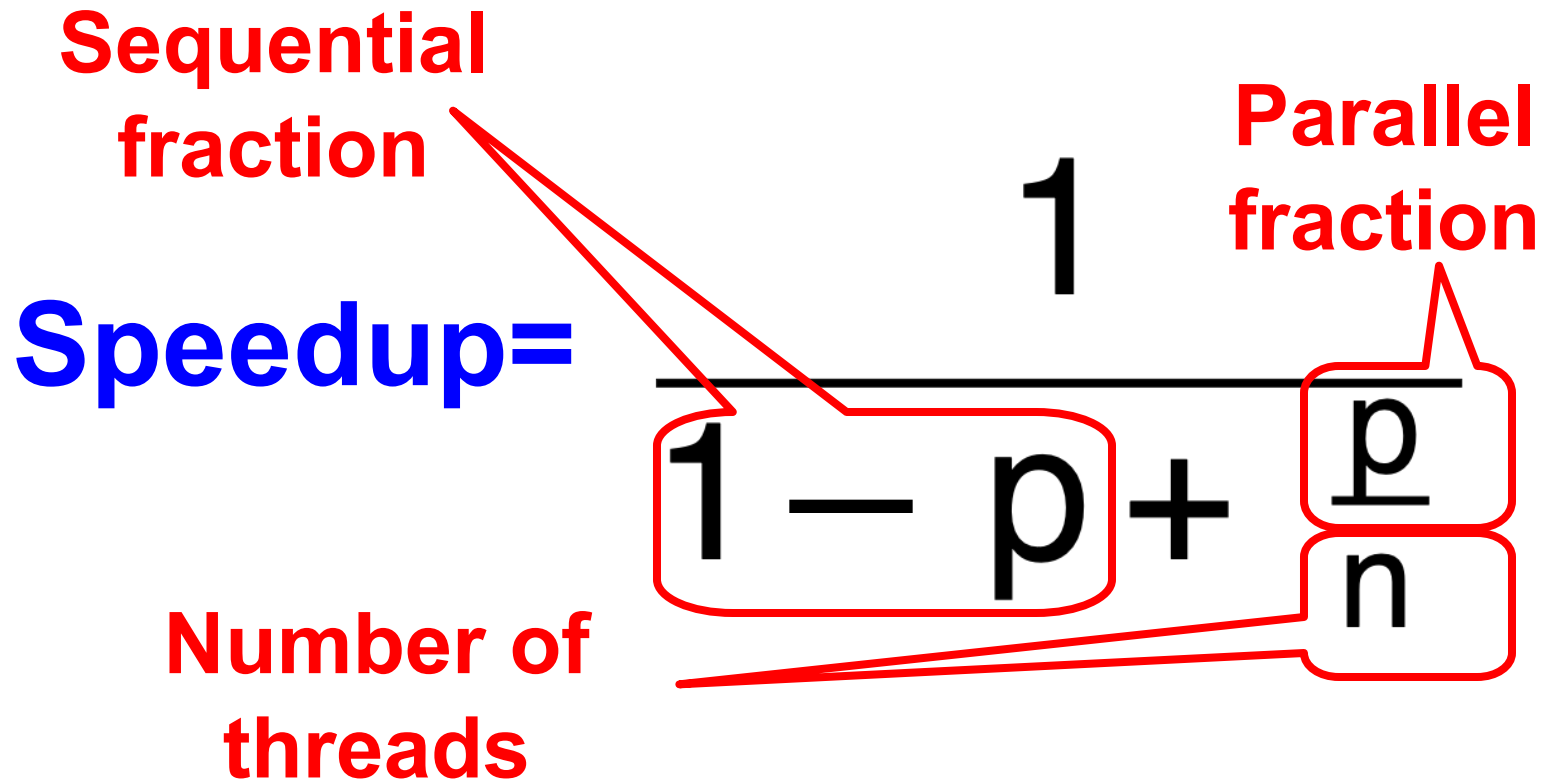
Sequential fraction

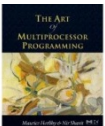
Speedup=

Parallel fraction

$$\frac{1}{1 - p + \frac{p}{n}}$$

Number of threads

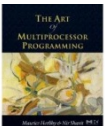




Example

- Ten processors
- 60% concurrent, 40% sequential
- How close to 10-fold speedup?

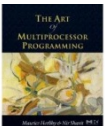
$$\text{Speedup} = 2.17 = \frac{1}{1 - 0.6 + \frac{0.6}{10}}$$



Example

- Ten processors
- 80% concurrent, 20% sequential
- How close to 10-fold speedup?

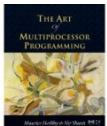
$$\text{Speedup} = 3.57 = \frac{1}{1 - 0.8 + \frac{0.8}{10}}$$



Example

- Ten processors
- 90% concurrent, 10% sequential
- How close to 10-fold speedup?

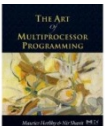
$$\text{Speedup} = 5.26 = \frac{1}{1 - 0.9 + \frac{0.9}{10}}$$



Example

- Ten processors
- 99% concurrent, 1% sequential
- How close to 10-fold speedup?

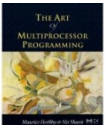
$$\text{Speedup} = 9.17 = \frac{1}{1 - 0.99 + \frac{0.99}{10}}$$



Example

Max Speedup ?
$$\frac{1}{1 - 0.9 + \frac{0.9}{10}}$$

Max Speedup?
$$\frac{1}{1 - 0.99 + \frac{0.99}{10}}$$



Principles of Mutual Exclusion

A real-world analogy



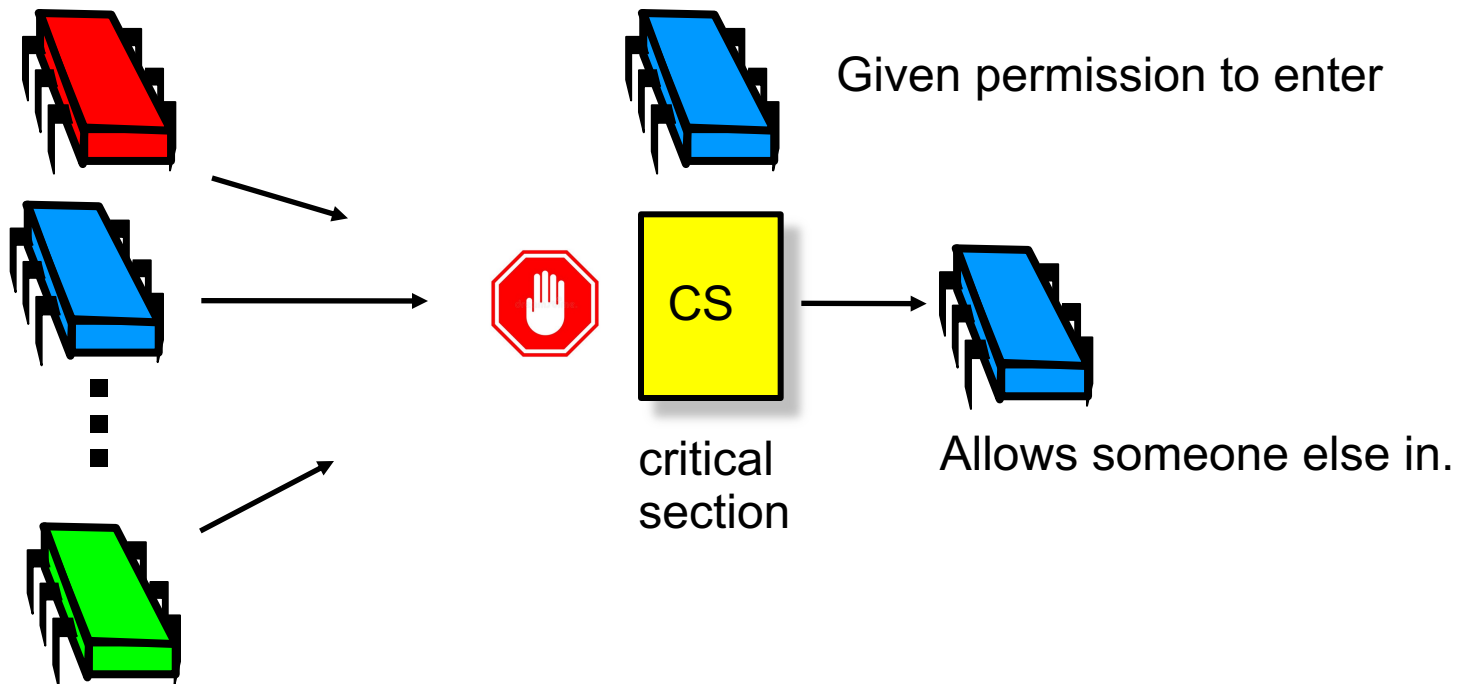
Critical sections, locks: Basic idea



<https://upload.wikimedia.org/wikipedia/commons/3/30/TokenRingSanta.jpg>

M3 Objective

- How we are going to ensure determinism when threads want to enter in the critical section?



Formalizing Critical Sections

- Synchronization primitives need to adhere to the following properties and principles about critical sections in order to be correct:
- **Mutual Exclusion Property:**
 - Only one thread is executing a critical section at a time
 - This is a safety property.
- **Deadlock-freedom property:**
 - If multiple threads simultaneously request to enter a critical section, then it must allow one to proceed
 - It implies the system never “freezes”.
 - This is a liveness property.
- **Starvation-freedom property:**
 - Every thread that attempts to acquire the lock eventually succeeds.
 - Also known as lockout freedom or bounded-waiting
 - This is a liveness property.
- **Fairness Principle:**
 - A thread who just left the critical section cannot immediately re-enter if other threads have already requested to enter the critical section.

Mutex in Go

- Go has a package called sync that provides basic synchronization primitives such as mutual exclusion locks.
 - The sync package Go's provides mutual exclusion with sync.Mutex and its two methods:
 - **m.Lock()**: locks m. If the lock is already in use, the calling goroutine blocks until the mutex is available.
 - **m.Unlock()**: Unlock unlocks m. It is a run-time error if m is not locked on entry to Unlock.
- Now let's take a look at how locks are implemented behind the scenes.

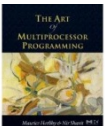
Practical Implementation of Locks

Demo/Thought experiment

- How to use a lock?
- How can we implement a lock?

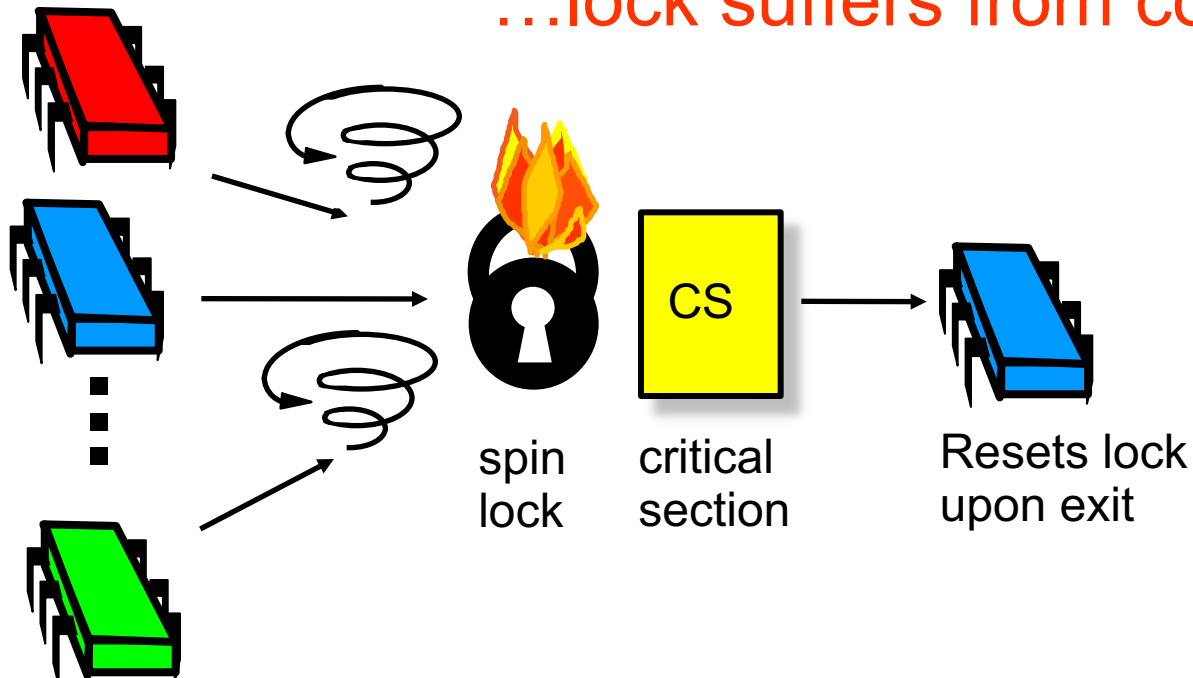
What Should you do if you can't get a lock?

- Keep trying
 - “spin” or “busy-wait”
 - When spinning, a thread repeatedly tests a condition, but, effectively, does no useful work until the condition has the appropriate value.
 - Can be very wasteful of CPU cycles.
 - Good if delays are short
- Give up the processor
 - Good if delays are long
 - Always good on uniprocessor

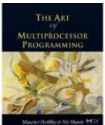


Basic Spin-Lock

...lock suffers from contention



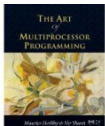
Seq Bottleneck → no parallelism



Implementing Locks

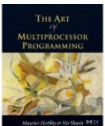
Two-Thread vs n -Thread Solutions

- 2-thread solutions first
 - Illustrate most basic ideas
 - Fits on one slide
- Then n -thread solution
- You will never use these protocols
 - Get over it
- You are advised to understand them
 - The same issues show up everywhere
 - Except hidden and more complex



Two-Thread Conventions

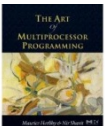
```
class ... implements Lock {  
    ...  
    // thread-local index, 0 or 1  
    public void lock() {  
        int i = ThreadID.get();  
        int j = 1 - i;  
    ...  
    }  
}
```



Two-Thread Conventions

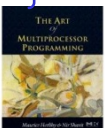
```
class ... implements Lock {  
    ...  
    // thread-local index, 0 or 1  
    public void lock() {  
        int i = ThreadID.get();  
        int j = 1 - i;  
        ...  
    }  
}
```

Henceforth: **i** is current
thread, **j** is other thread



LockOne

```
class LockOne implements Lock {  
    private boolean[] flag = new boolean[2];  
    public void lock() {  
        int i = ThreadID.get();  
        int j = 1 - i;  
        flag[i] = true;  
        while (flag[j]) {}  
    }  
    public void unlock() {  
        int i = ThreadID.get();  
        flag[i] = false;  
    }  
}
```



LockOne

```
class LockOne implements Lock {  
    private boolean[] flag = new boolean[2];  
    public void lock() {  
        flag[i] = true;  
        while (flag[j]) {}  
    }  
}
```

Each thread has flag

LockOne

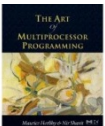
```
class LockOne implements Lock {  
    private boolean[] flag = new boolean[2];  
    public void lock() {  
        flag[i] = true;  
        while (flag[j]) {}  
    }  
}
```

Set my flag

LockOne

```
class LockOne implements Lock {  
    private boolean[] flag = new boolean[2];  
    public void lock() {  
        flag[i] = true;  
        while (flag[j]) {}  
    }  
}
```

Wait for other flag to become
false

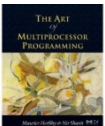


Deadlock Freedom

- LockOne Fails deadlock-freedom
 - Concurrent execution can deadlock

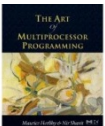
```
flag[i] = true;    flag[j] = true;
while (flag[j]){}  while (flag[i]){}
```

- Sequential executions OK



LockTwo

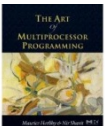
```
public class LockTwo implements Lock {  
    private int victim;  
    public void lock() {  
        int i = ThreadID.get();  
        int j = 1 - i;  
        victim = i;  
        while (victim == i) {}  
    }  
  
    public void unlock() {}  
}
```



LockTwo

```
public class LockTwo implements Lock {  
    private int victim;  
    public void lock() {  
        victim = i;  
        while (victim == i) {}  
    }  
  
    public void unlock() {}  
}
```

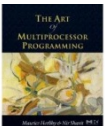
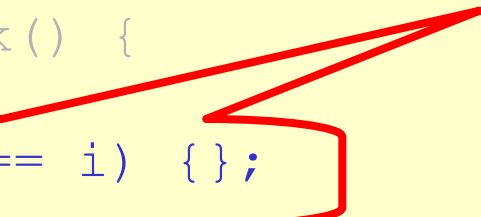
Let other go first



LockTwo

```
public class LockTwo implements Lock {  
    private int victim;  
    public void lock() {  
        victim = i;  
        while (victim == i) {};  
    }  
  
    public void unlock() {}  
}
```

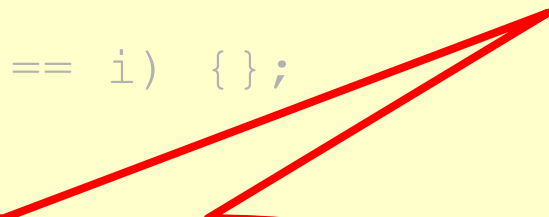
Wait for
permission



LockTwo

```
public class Lock2 implements Lock {  
    private int victim;  
    public void lock() {  
        victim = i;  
        while (victim == i) {};  
    }  
  
    public void unlock() {}  
}
```

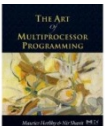
Nothing to do



LockTwo Claims

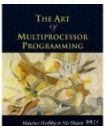
- Satisfies mutual exclusion
 - If thread **i** in critical section
 - Then **victim == j**
 - Cannot be both 0 and 1
- Not deadlock free
 - Sequential execution deadlocks
 - Concurrent execution does not

```
public void LockTwo() {  
    victim = i;  
    while (victim == i) {};  
}
```



Peterson's Algorithm

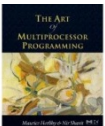
```
public void lock() {  
    int i = ThreadID.get();  
    int j = 1 - i;  
    flag[i] = true;  
    victim = i;  
    while (flag[j] && victim == i) {};  
}  
public void unlock() {  
    flag[i] = false;  
}
```



Peterson's Algorithm

Announce I'm
interested

```
public void lock() {  
    flag[i] = true;  
    victim = i;  
    while (flag[j] && victim == i) {}  
}  
public void unlock() {  
    flag[i] = false;  
}
```

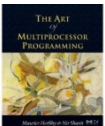


Peterson's Algorithm

```
public void lock() {  
    flag[i] = true;  
    victim = i;  
    while (flag[j] && victim == i) {}  
}  
public void unlock() {  
    flag[i] = false;  
}
```

Announce I'm
interested

Defer to other



Peterson's Algorithm

```
public void lock() {
```

```
    flag[i] = true;
```

```
    victim = i;
```

```
    while (flag[j] && victim == i) {};
```

```
}
```

```
public void unlock() {
```

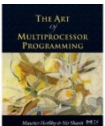
```
    flag[i] = false;
```

```
}
```

Announce I'm
interested

Defer to other

Wait while other
interested & I'm the
victim



Peterson's Algorithm

```
public void lock() {
```

```
    flag[i] = true;
```

```
    victim = i;
```

```
    while (flag[j] && victim == i) {};
```

```
}
```

```
public void unlock() {
```

```
    flag[i] = false;
```

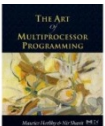
```
}
```

Announce I'm
interested

Defer to other

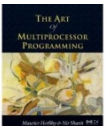
Wait while other
interested & I'm the
victim

No longer
interested



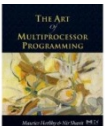
Bakery Algorithm

- N-threaded locking algorithm
- Provides First-Come-First-Served
- How?
 - Take a “number”
 - Wait until lower numbers have been served
- Lexicographic order
 - $(a,i) > (b,j)$
 - If $a > b$, or $a = b$ and $i > j$



Bakery Algorithm

```
class Bakery implements Lock {  
    boolean[] flag;  
    Label[] label;  
    public Bakery (int n) {  
        flag = new boolean[n];  
        label = new Label[n];  
        for (int i = 0; i < n; i++) {  
            flag[i] = false; label[i] = 0;  
        }  
    }  
    ...  
}
```



Bakery Algorithm

```
class Bakery implements Lock {
```

```
    boolean[] flag;
```

```
    Label[] label;
```

```
    public Bakery (int n) {
```

```
        flag = new boolean[n];
```

```
        label = new Label[n];
```

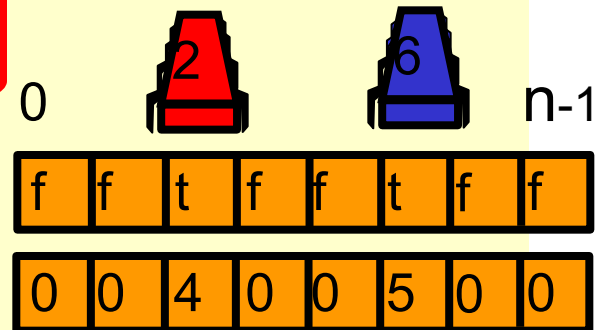
```
        for (int i = 0; i < n; i++) {
```

```
            flag[i] = false; label[i] = 0;
```

```
        }
```

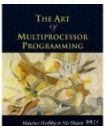
```
    }
```

```
    ...
```



Bakery Algorithm

```
class Bakery implements Lock {  
    ...  
    public void lock() {  
        flag[i] = true;  
        label[i] = max(label[0], ..., label[n-1])+1;  
        while ( $\exists k$  flag[k]  
                && (label[i],i) > (label[k],k));  
    }
```



Bakery Algorithm

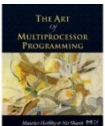
```
class Bakery implements Lock {  
    ...  
    public void lock() {  
        flag[i] = true;  
        label[i] = max(label[0], ..., label[n-1])+1;  
        while ( $\exists k$  flag[k]  
                && (label[i], i) > (label[k], k));  
    }  
}
```

I'm interested

Bakery Algorithm

Take increasing
label (read labels in
some arbitrary
order)

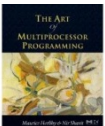
```
class Bakery implements Lock {  
    ...  
    public void lock() {  
        flag[i] = true;  
        label[i] = max(label[0], ..., label[n-1]) + 1;  
        while ( $\exists k$  flag[k]  
                && (label[i], i) > (label[k], k));  
    }  
}
```



Bakery Algorithm

```
class Bakery implements Lock {  
    ...  
    public void lock() {  
        flag[i] = true;  
        label[i] = max(label[0], ..., label[n-1])+1;  
        while ( $\exists k$  flag[k]  
                && (label[i], i) > (label[k], k));  
    }
```

Someone is
interested

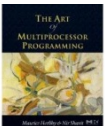


Bakery Algorithm

```
class Bakery implements Lock {  
    boolean flag[n];  
    int label[n];  
  
    public void lock() {  
        flag[i] = true;  
        label[i] = max(label[0], ..., label[n-1])+1;  
        while ( $\exists k$  flag[k]  
                && (label[i], i) > (label[k], k));  
    }
```

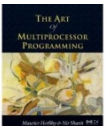
Someone is
interested ...

... whose (label,i) in
lexicographic order is lower



Bakery Algorithm

```
class Bakery implements Lock {  
  
    ...  
  
    public void unlock() {  
        flag[i] = false;  
    }  
}
```

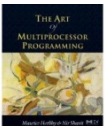


Bakery Algorithm

```
class Bakery implements Lock {  
    ...  
  
    public void unlock() {  
        flag[i] = false;  
    }  
}
```

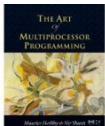
No longer
interested

labels are always increasing



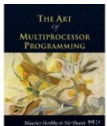
Deep Philosophical Question

- The Bakery Algorithm is
 - Succinct,
 - Elegant, and
 - Fair.
- Q: So why isn't it practical?
- A: Well, you have to read **N** distinct variables



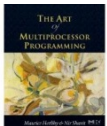
Bakery Algorithm

- Has no deadlock and adheres to mutual exclusion property
- There is always one thread with earliest label
- Is there a problem with the labeling portion of the algorithm?
- Is there another problem?



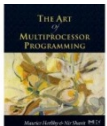
Test-and-Set

- Boolean value
- Test-and-set (TAS)
 - Set to **true** and return current value
 - Return value tells if prior value was **true** or **false**
- Can reset just by writing **false**
- TAS aka “getAndSet”



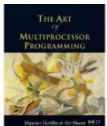
Test-and-Set

```
public class AtomicBoolean {  
    boolean value;  
  
    public synchronized boolean  
        getAndSet(boolean newValue) {  
        boolean prior = value;  
        value = newValue;  
        return prior;  
    }  
}
```



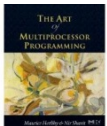
Test-and-Set Locks

- Locking
 - Lock is free: value is false
 - Lock is taken: value is true
- Acquire lock by calling TAS
 - If result is false, you win
 - If result is true, you lose
- Release lock by writing false



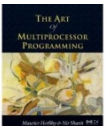
Test-and-set Lock

```
class TASlock {  
    AtomicBoolean state =  
        new AtomicBoolean(false);  
  
    void lock() {  
        while (state.getAndSet(true)) {}  
    }  
  
    void unlock() {  
        state.set(false);  
    }  
}
```



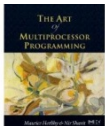
Space Complexity

- TAS spin-lock has small “footprint”
- N thread spin-lock uses $O(1)$ space
- As opposed to $O(n)$ Peterson/Bakery

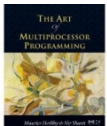
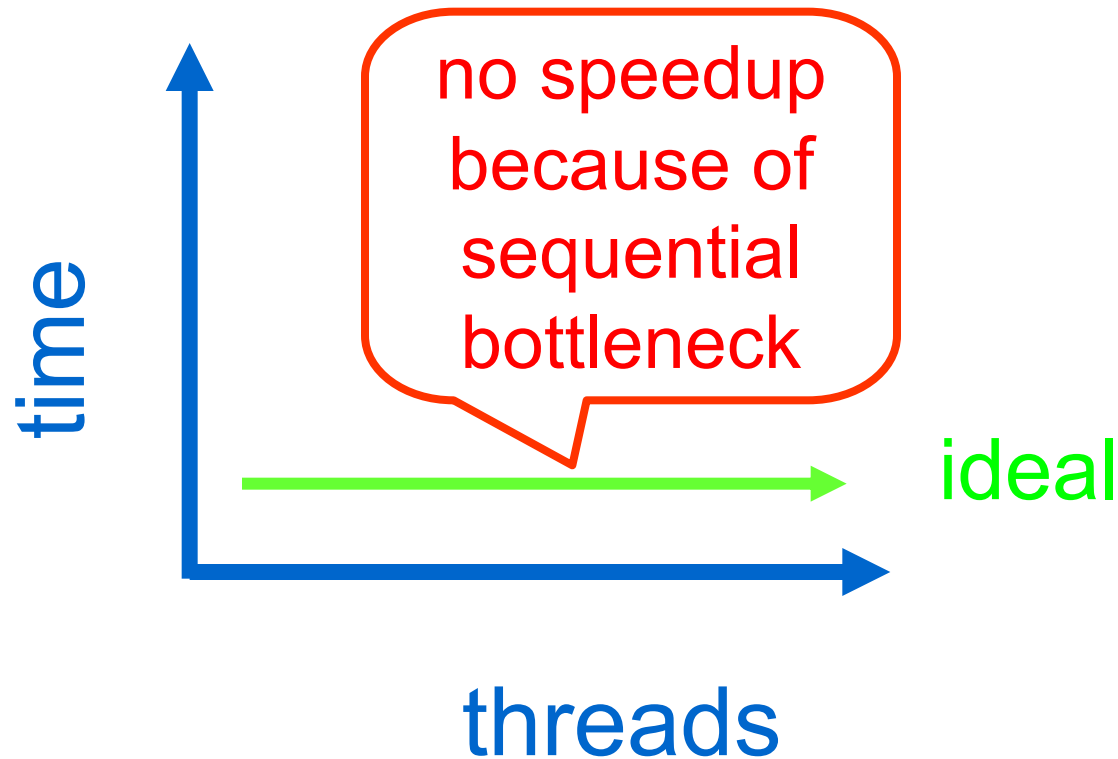


Performance

- Experiment
 - n threads
 - Increment shared counter 1 million times
- How long should it take?
- How long does it take?



Graph



Compare and Swap

```
function cas(p: pointer to int, old: int, new: int) is
    if *p ≠ old
        return false

    *p ← new

    return true
```

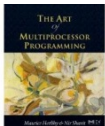
CAS is the generalized version of TAS: Not just booleans.

<https://en.wikipedia.org/wiki/Compare-and-swap>

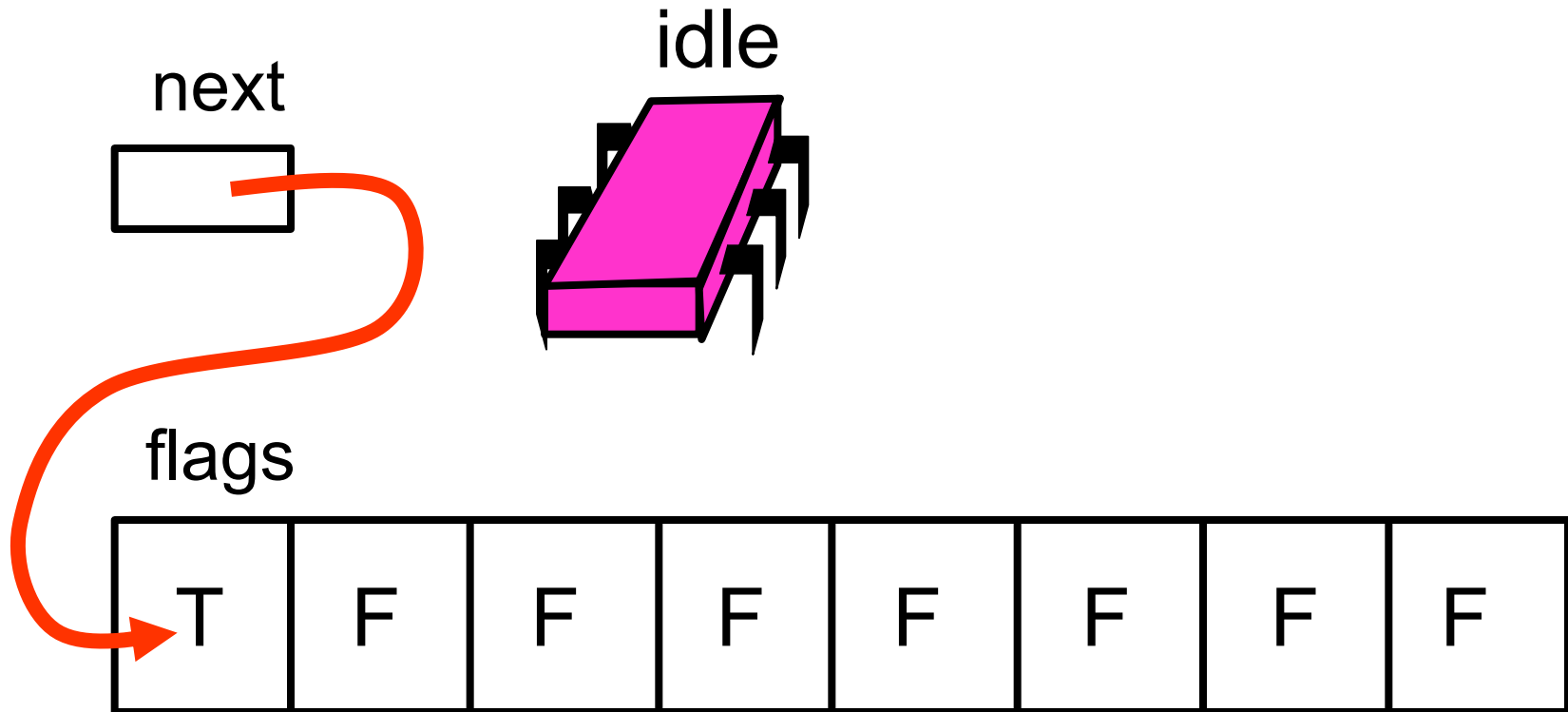
Fairness and Efficient Locks

Idea

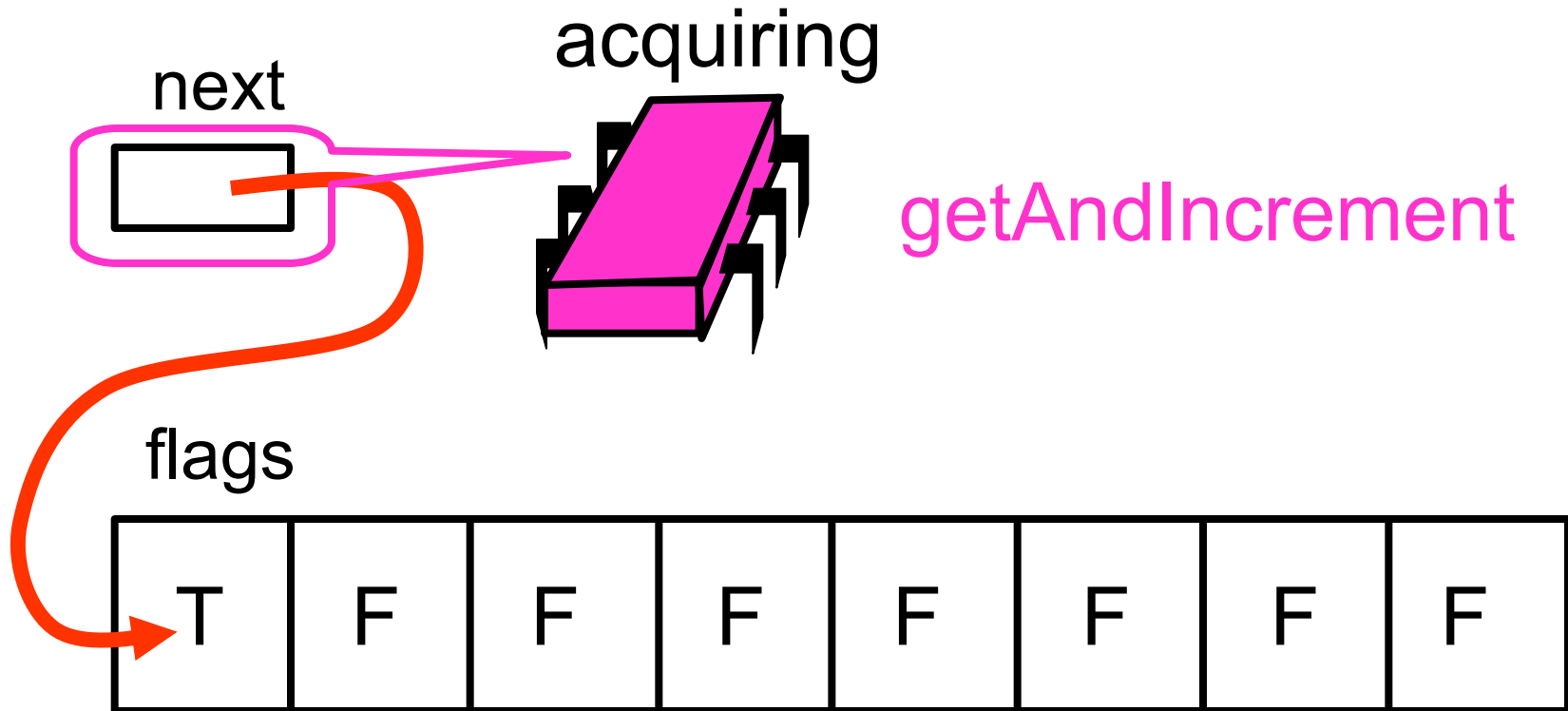
- Avoid useless invalidations
 - By keeping a queue of threads
- Each thread
 - Notifies next in line
 - Without bothering the others
 - Fairness when it comes to who should get lock next based on who arrived first.



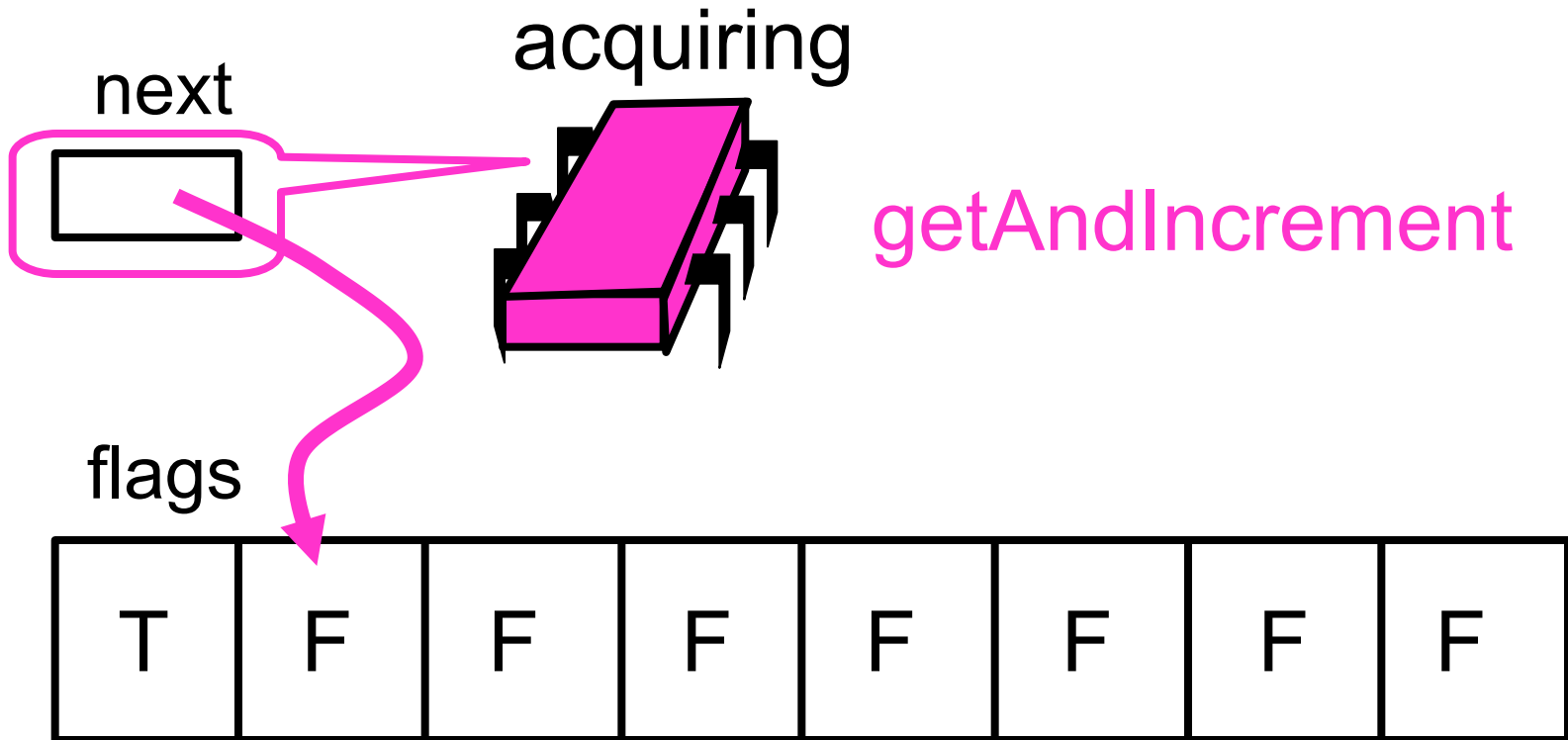
Anderson Queue Lock



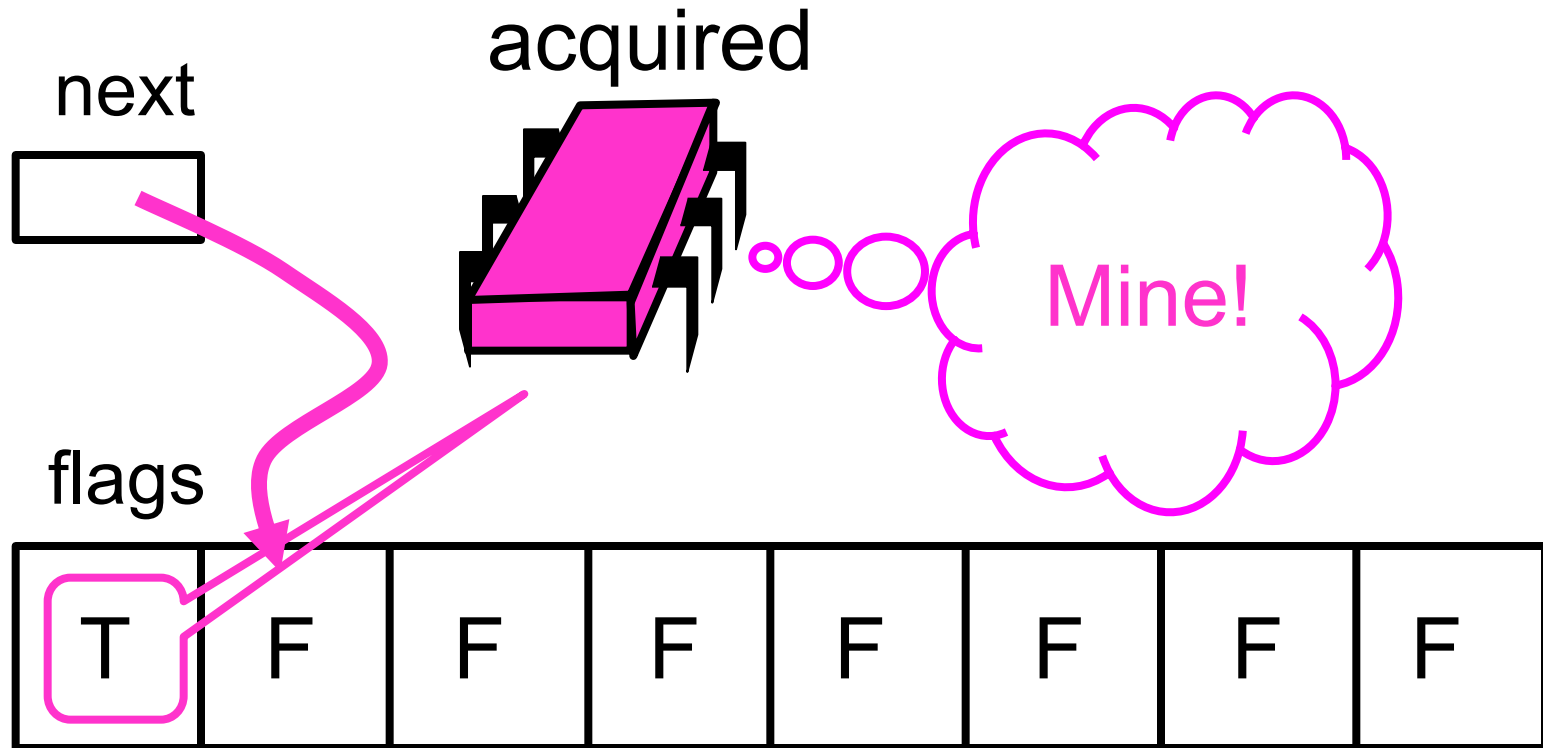
Anderson Queue Lock



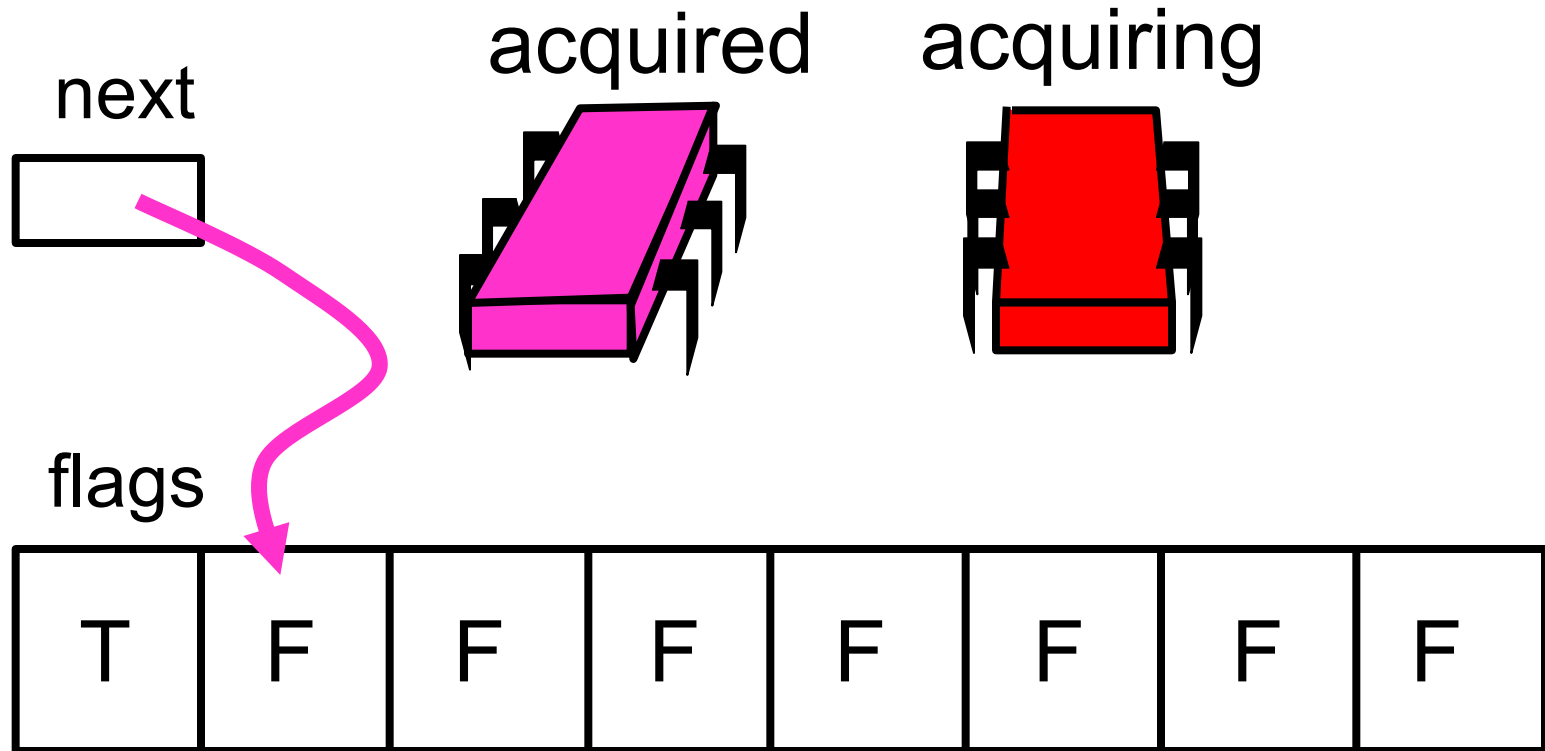
Anderson Queue Lock



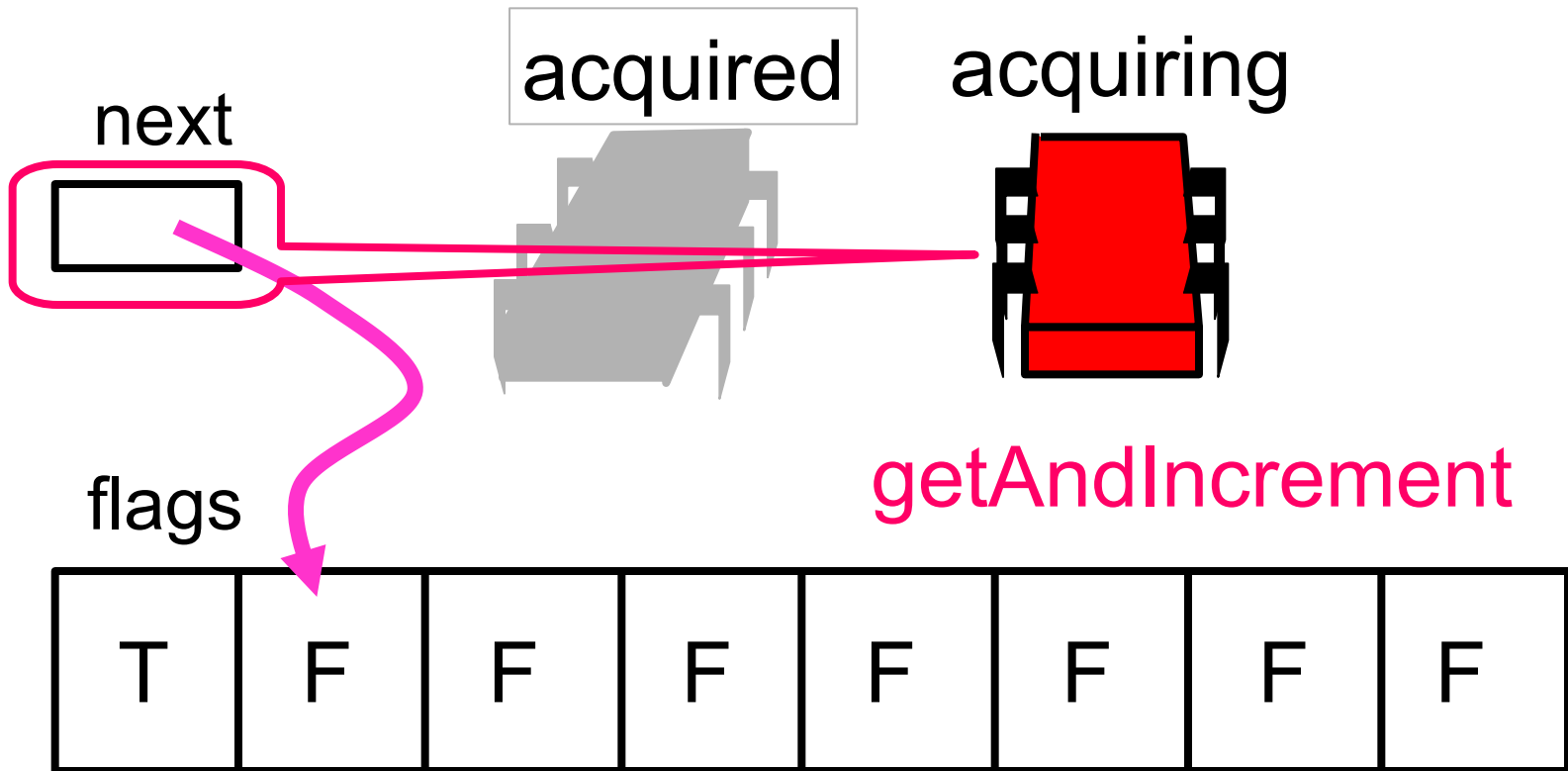
Anderson Queue Lock



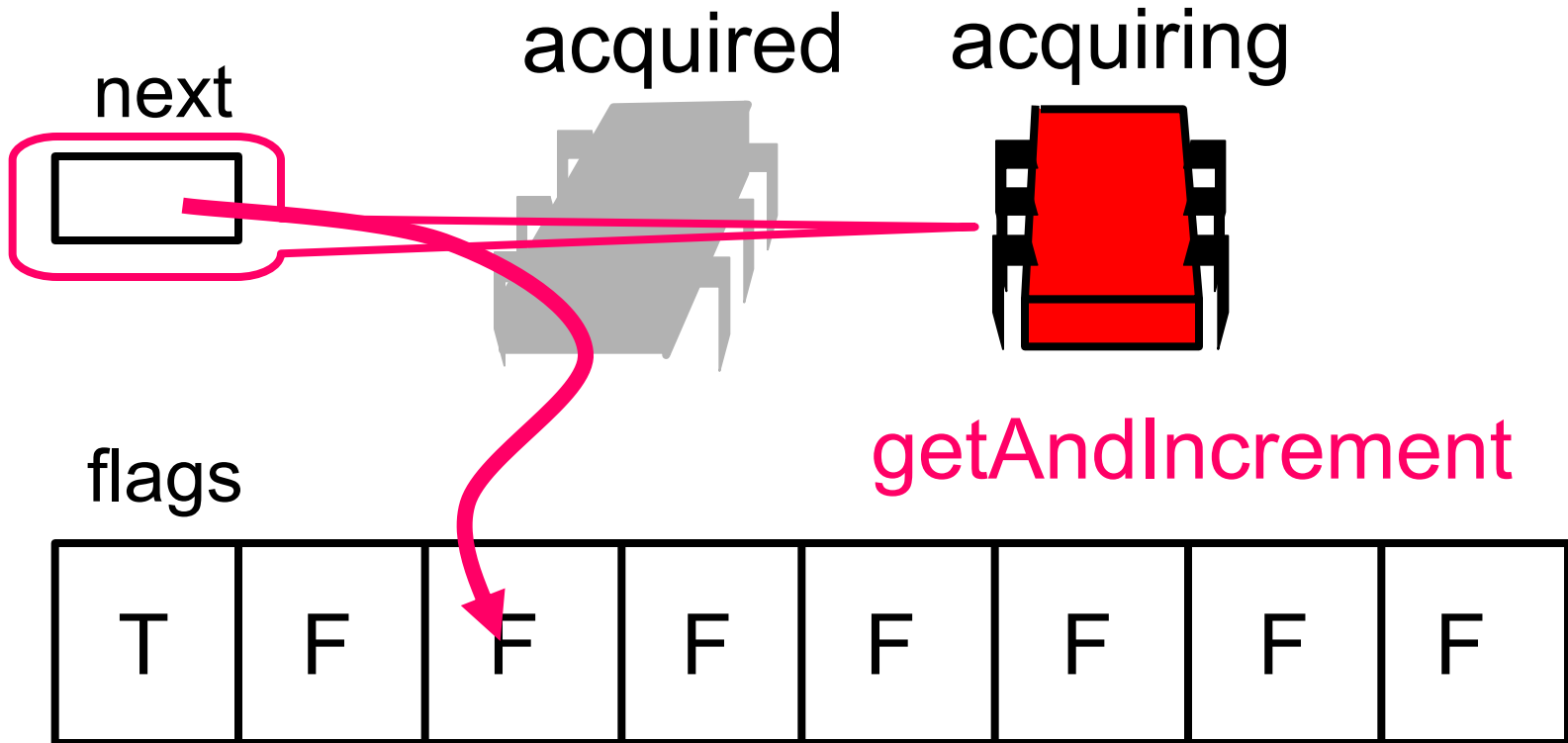
Anderson Queue Lock



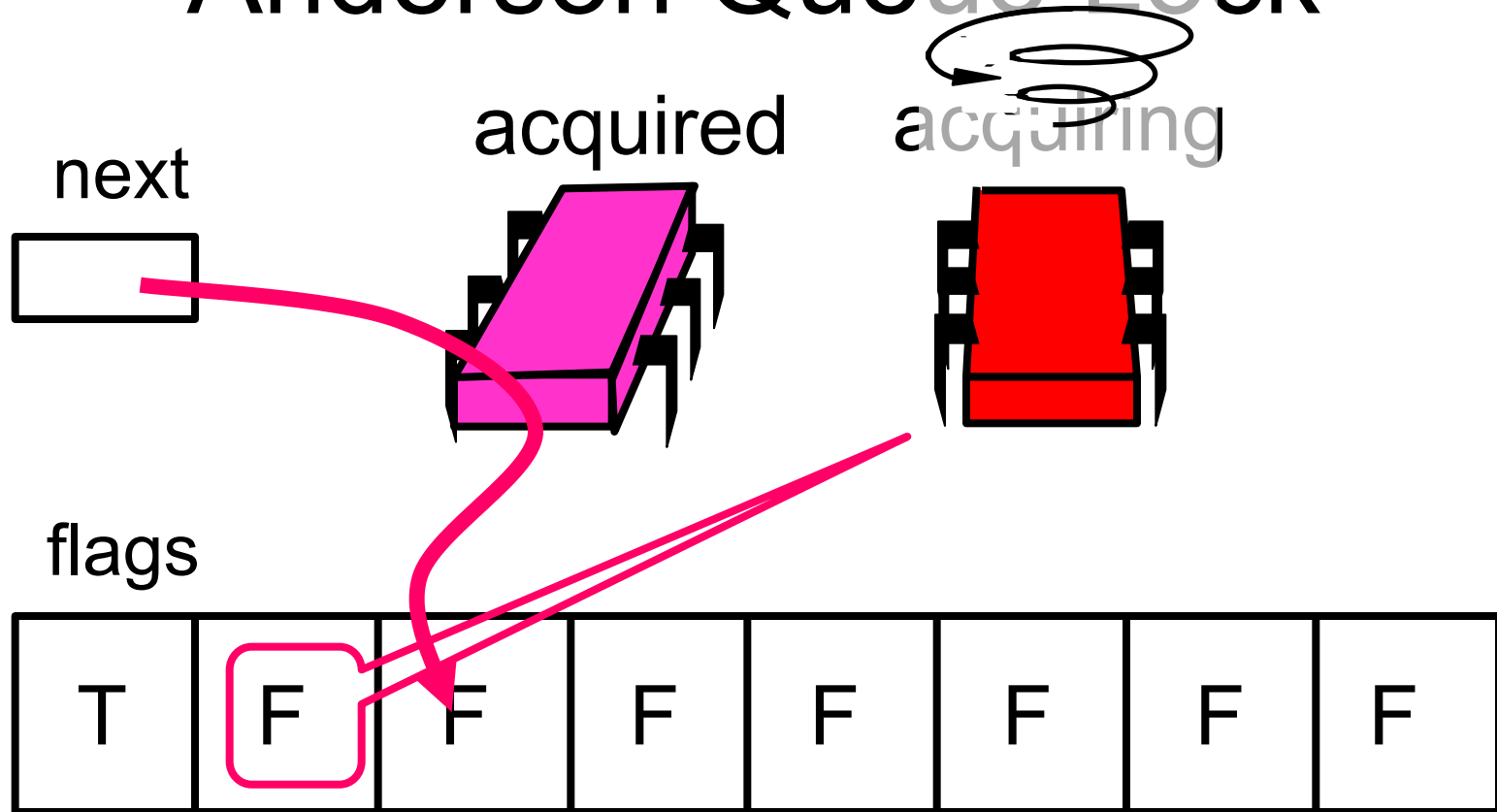
Anderson Queue Lock



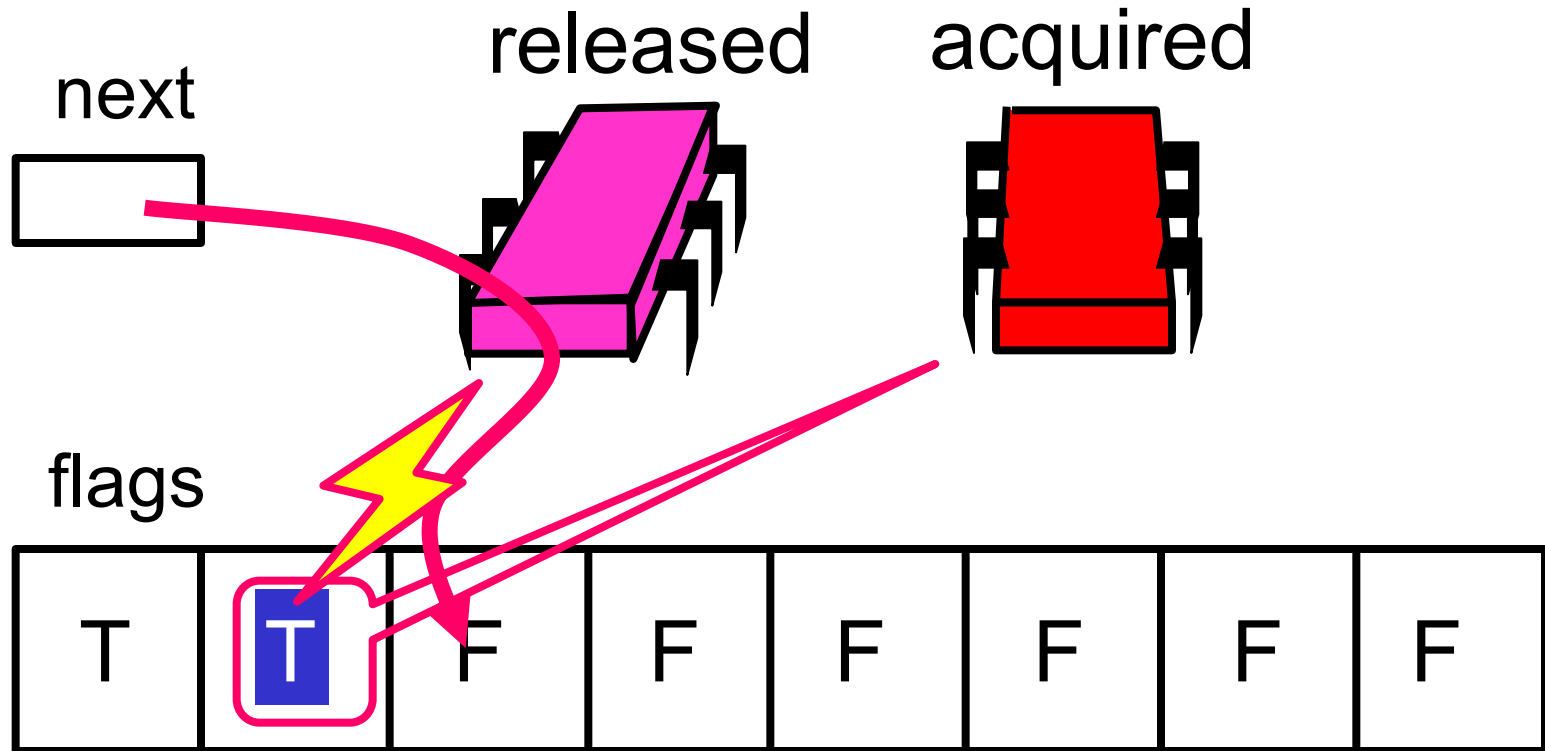
Anderson Queue Lock



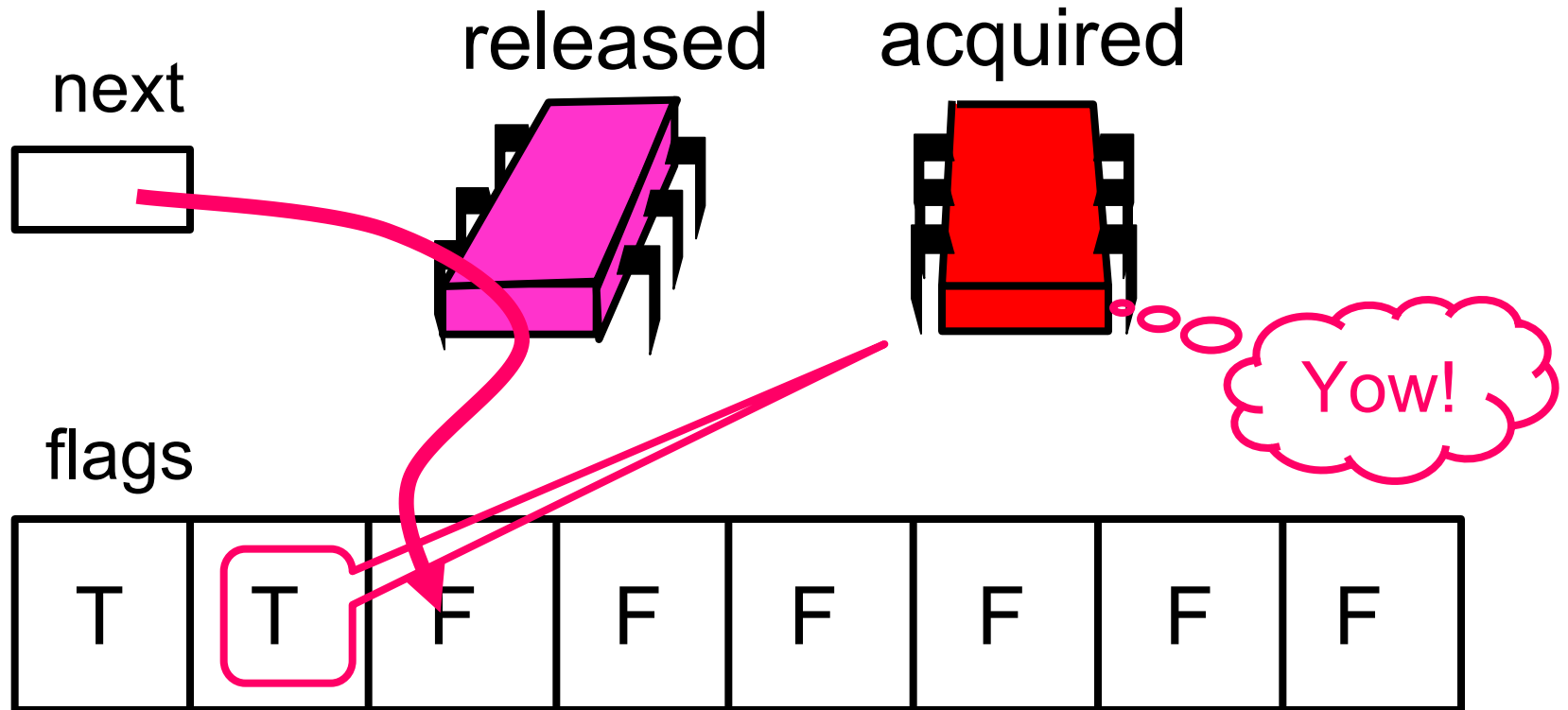
Anderson Queue Lock



Anderson Queue Lock

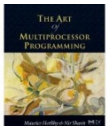


Anderson Queue Lock



Anderson Queue Lock

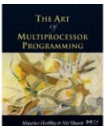
```
class ALock implements Lock {  
    boolean[] flags={true,false,...,false};  
    AtomicInteger next  
        = new AtomicInteger(0);  
    ThreadLocal<Integer> mySlot;
```



Anderson Queue Lock

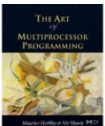
```
class ALock implements Lock {  
    boolean[] flags={true,false,...,false};  
    AtomicInteger next  
        = new AtomicInteger(0);  
    ThreadLocal<Integer> mySlot;
```

Thread-local variable

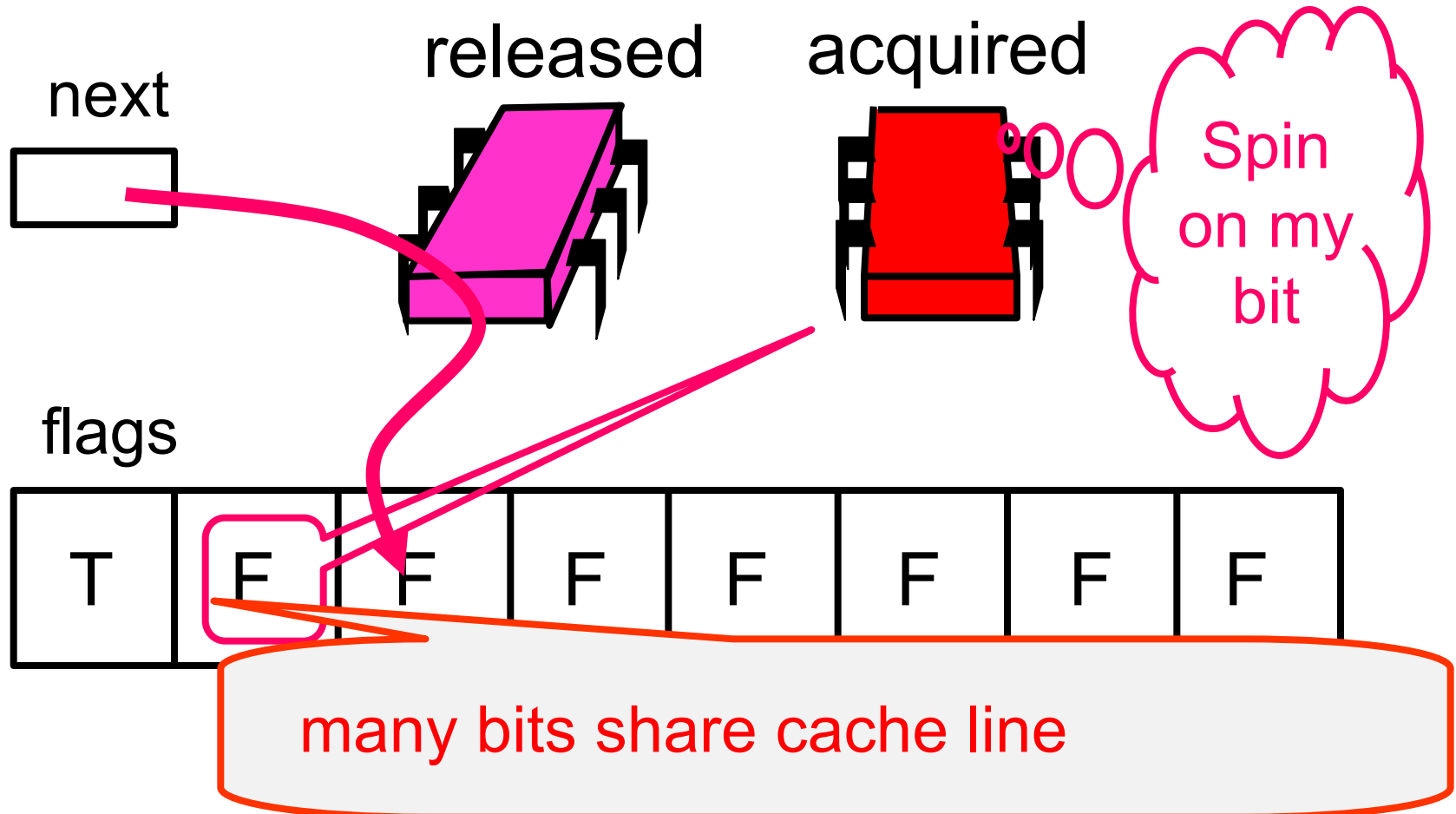


Anderson Queue Lock

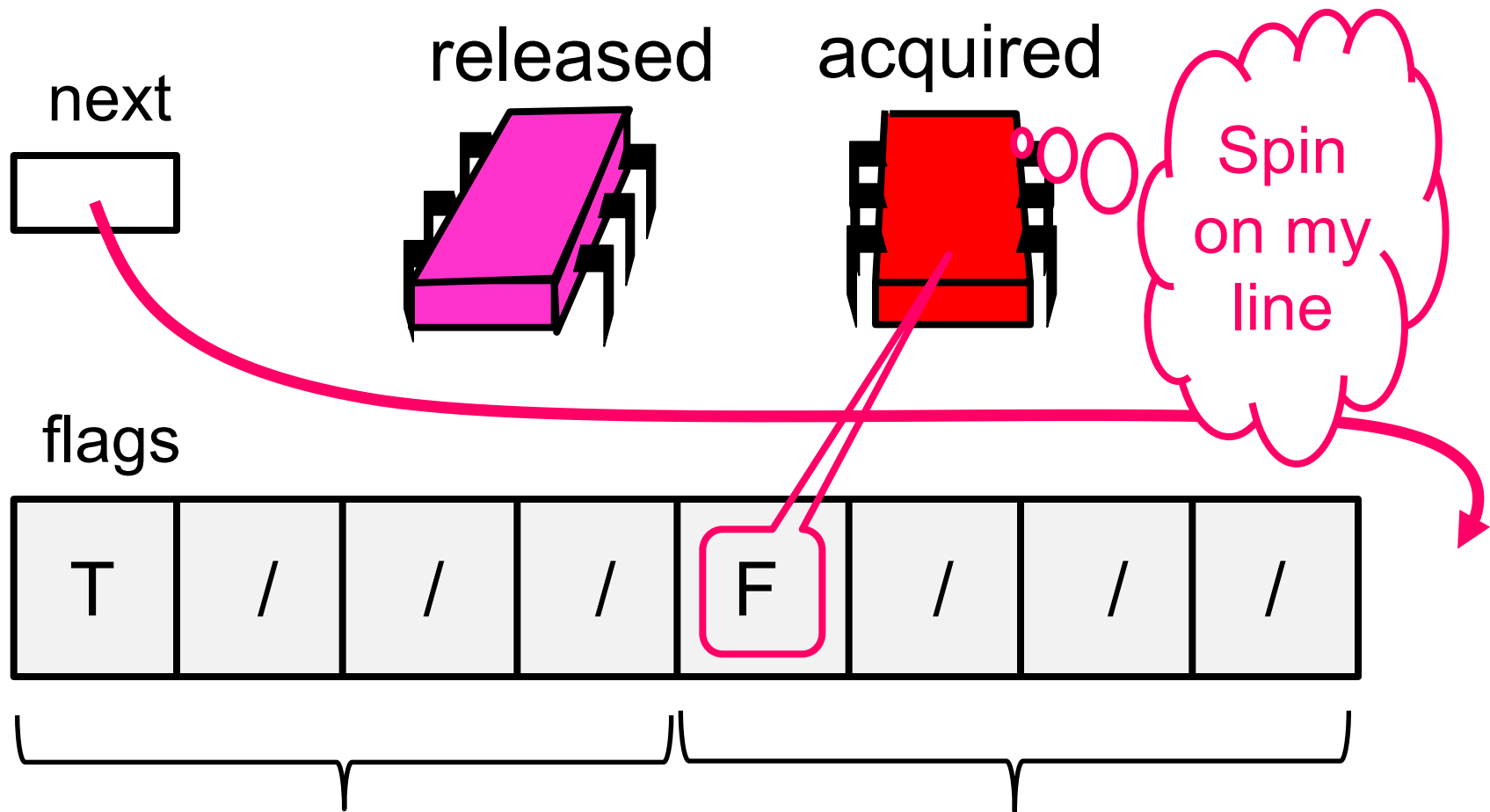
```
public lock() {  
    mySlot = next.getAndIncrement();  
    while (!flags[mySlot % n]) {};  
    flags[mySlot % n] = false;  
}  
  
public unlock() {  
    flags[(mySlot+1) % n] = true;  
}
```



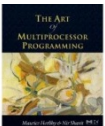
Local Spinning



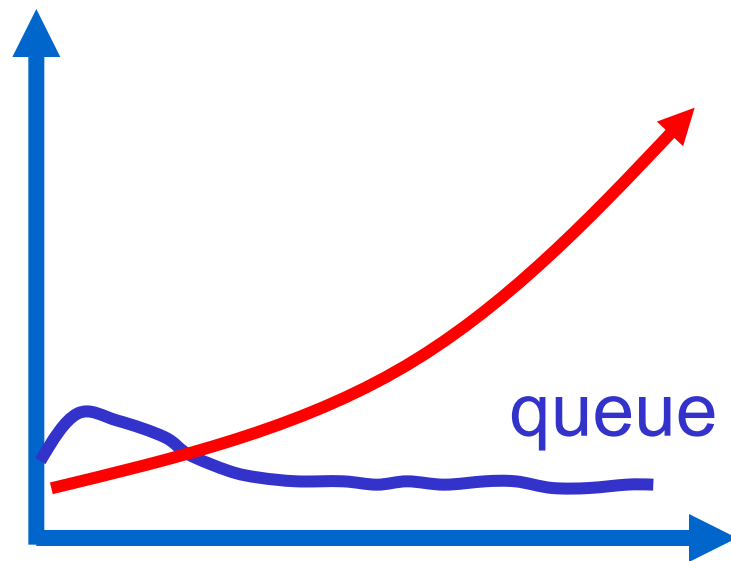
The Solution: Padding



Line 1 Art of Multiprocessor Programming Line 26

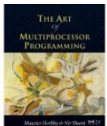


Performance



TAS

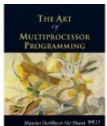
- Shorter handover than backoff
- Curve is practically flat
- Scalable performance



Anderson Queue Lock

Good

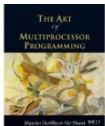
- First truly scalable lock
- Simple, easy to implement
- Back to FIFO order



Anderson Queue Lock

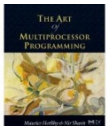
Bad

- Space hog...
- One bit per thread → one cache line per thread
 - What if unknown number of threads?
 - What if small number of actual contenders?

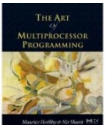
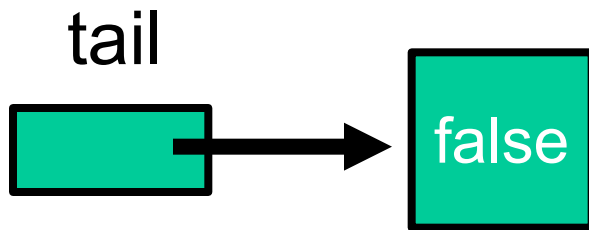
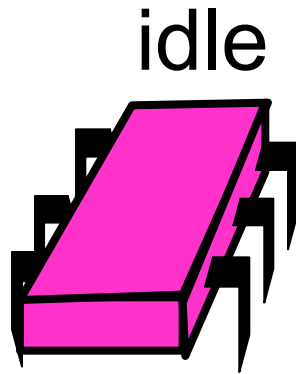


CLH Lock

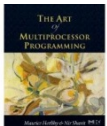
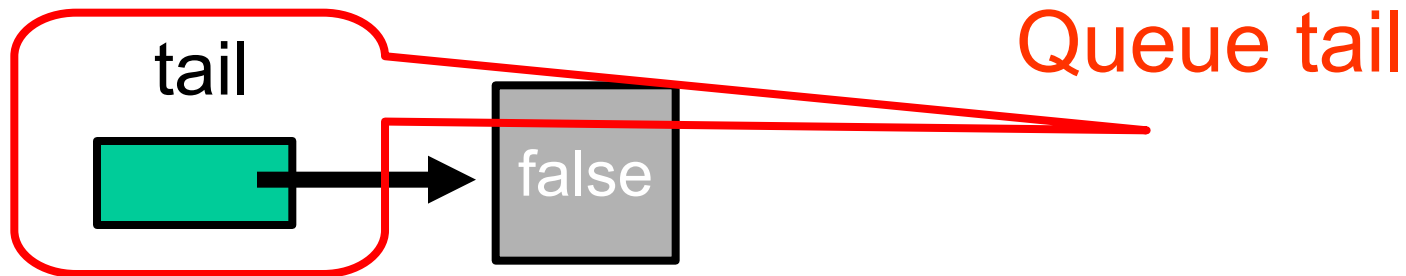
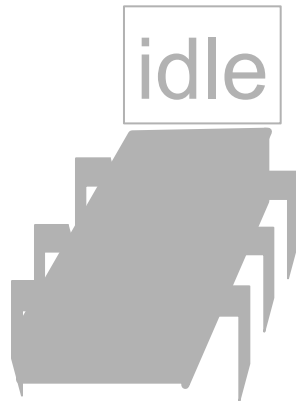
- FIFO order
- Small, constant-size overhead per thread



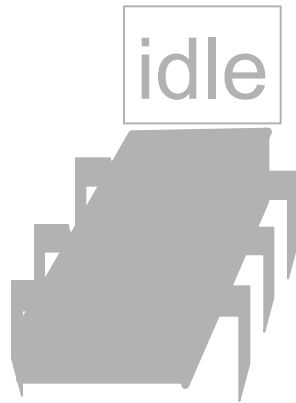
Initially



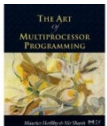
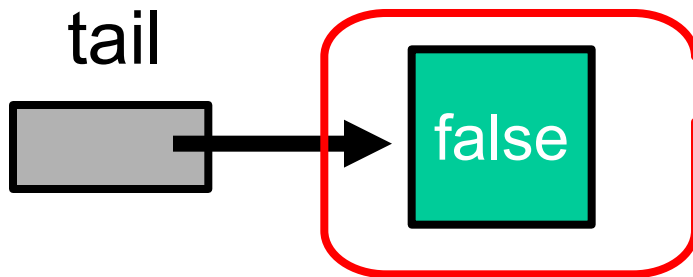
Initially



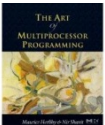
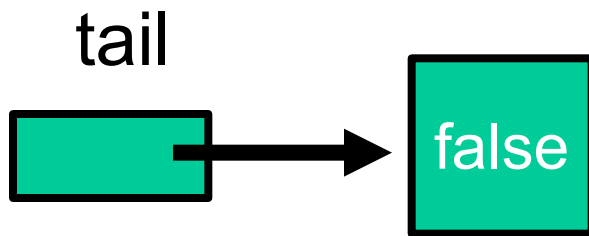
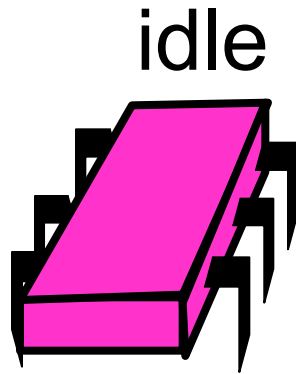
Initially



Lock is free

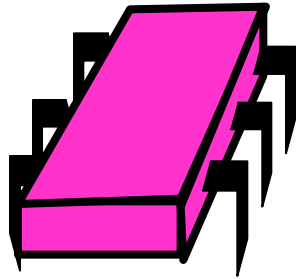


Initially

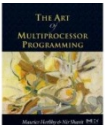
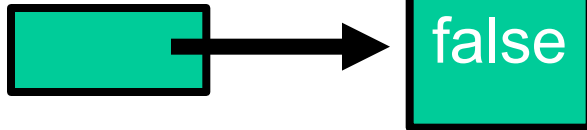


Purple Wants the Lock

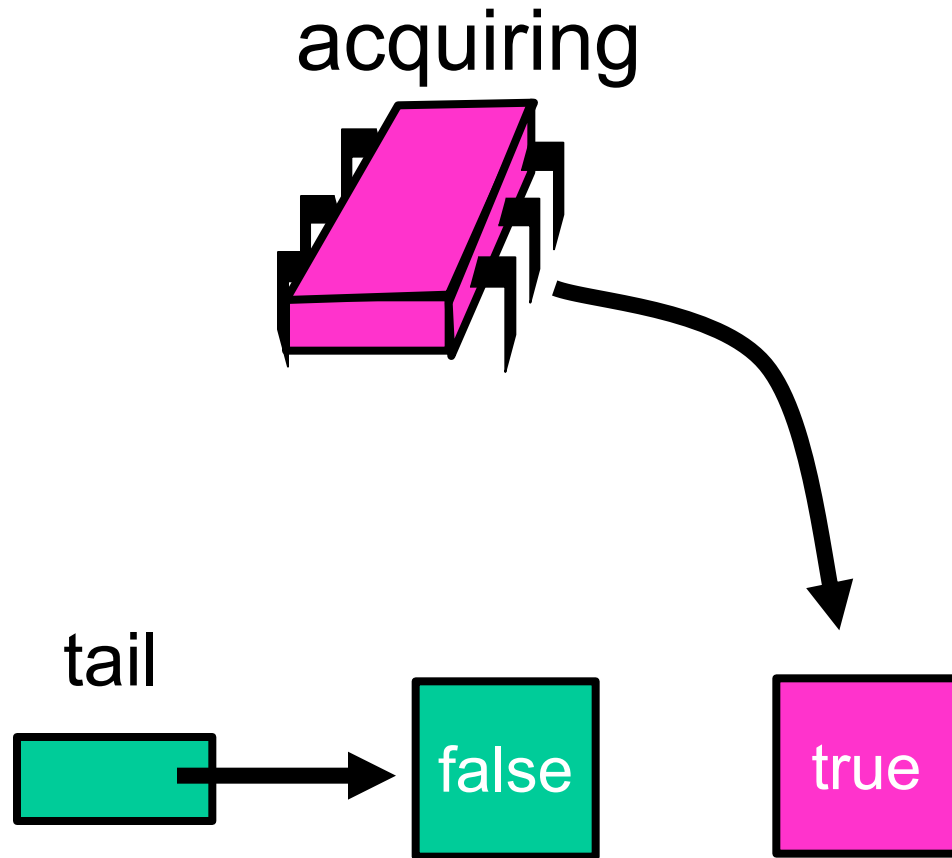
acquiring



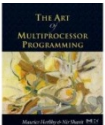
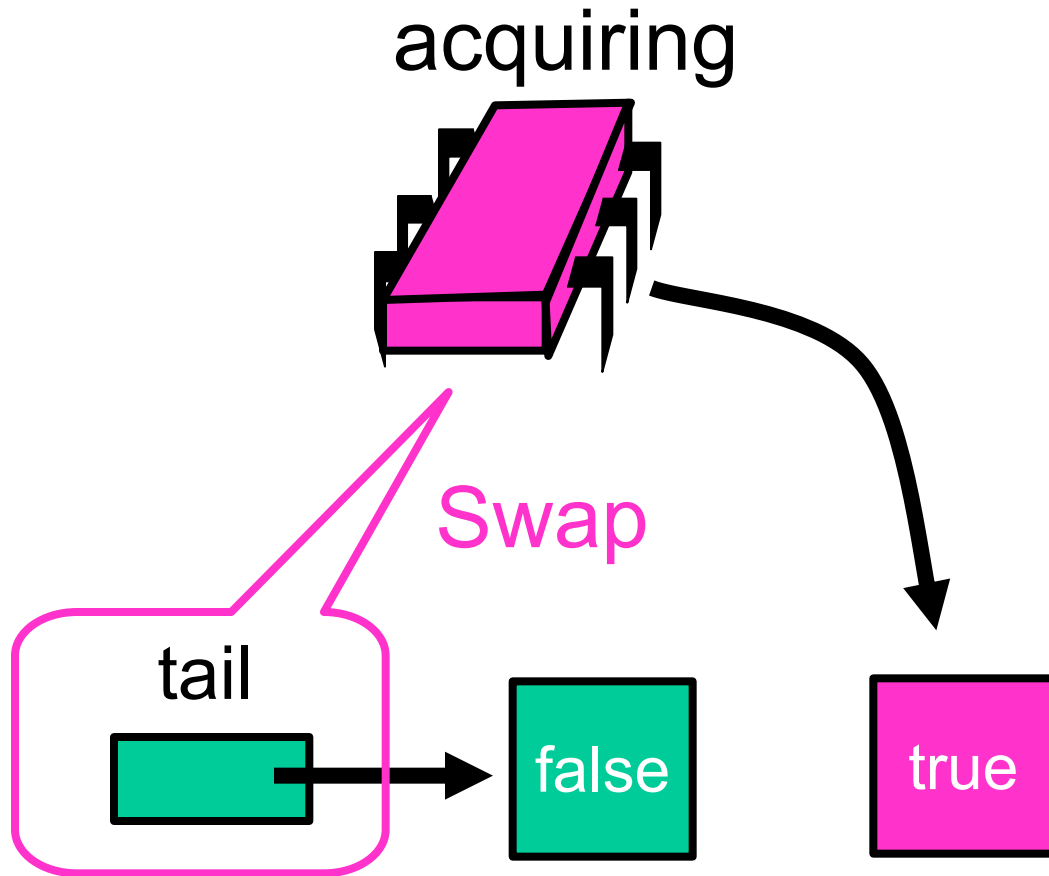
tail



Purple Wants the Lock

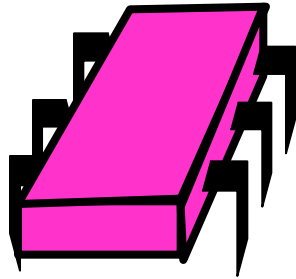


Purple Wants the Lock

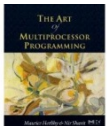
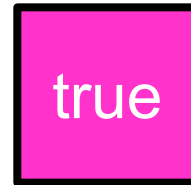
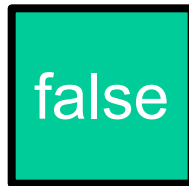


Purple Has the Lock

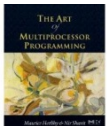
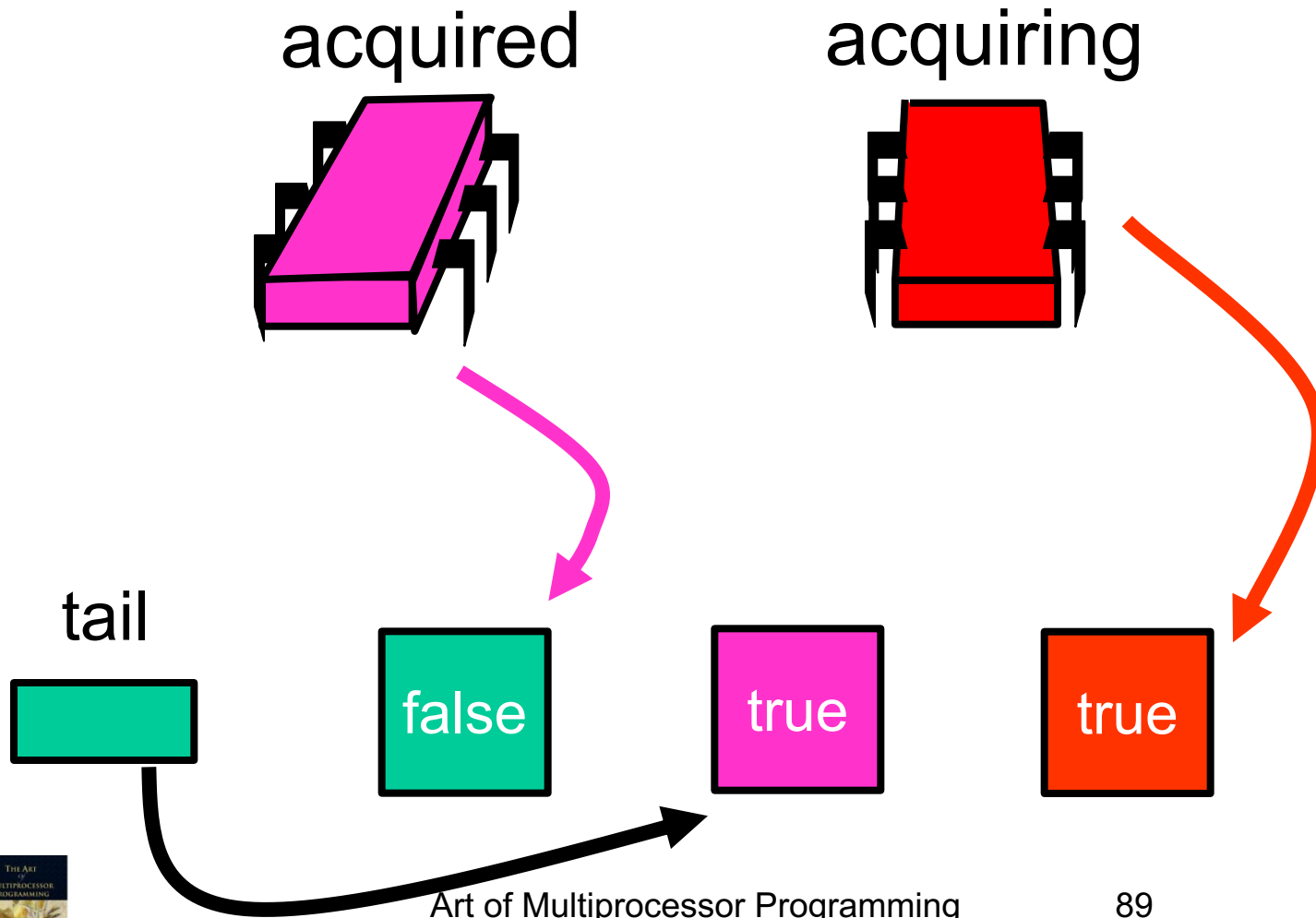
acquired



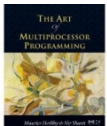
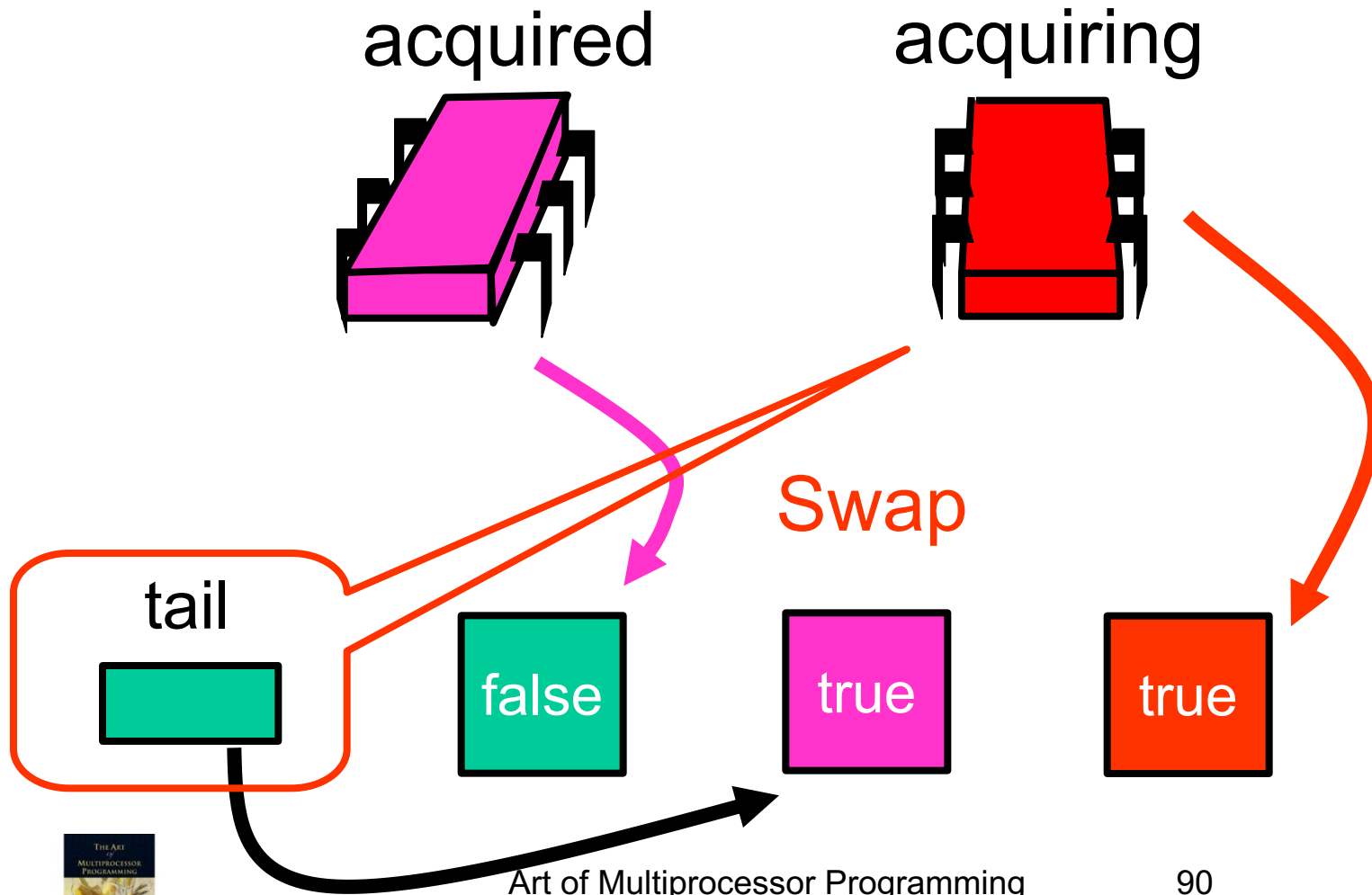
tail



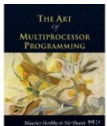
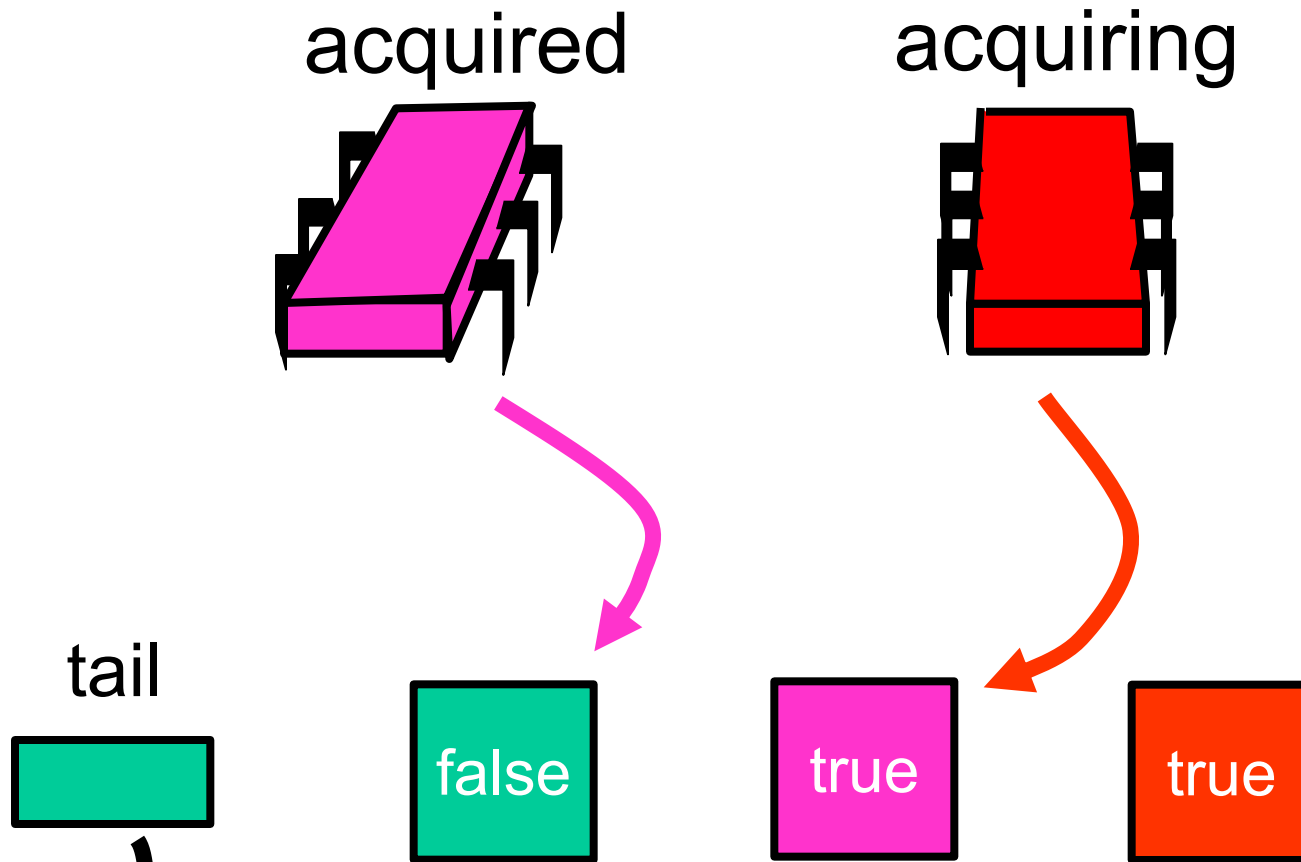
Red Wants the Lock



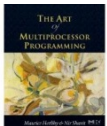
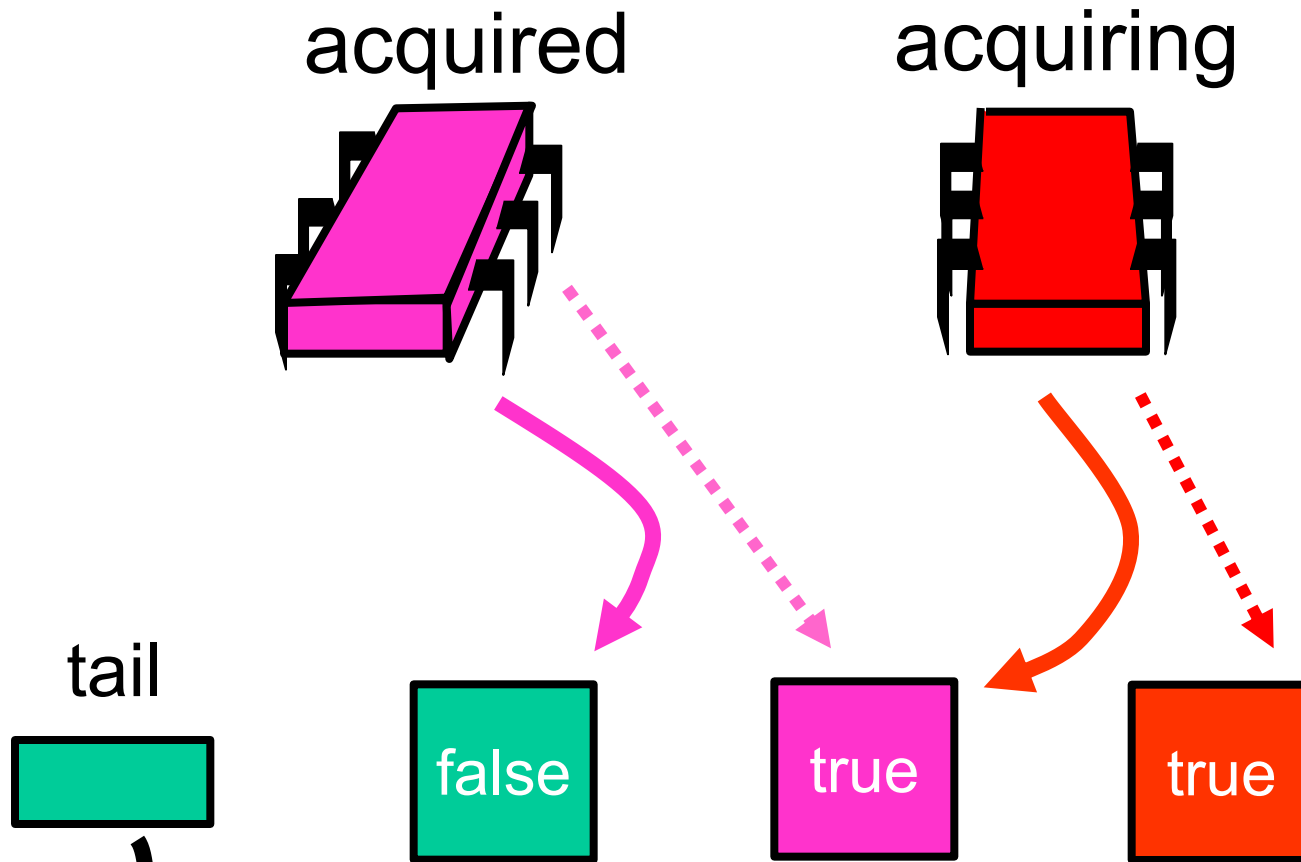
Red Wants the Lock



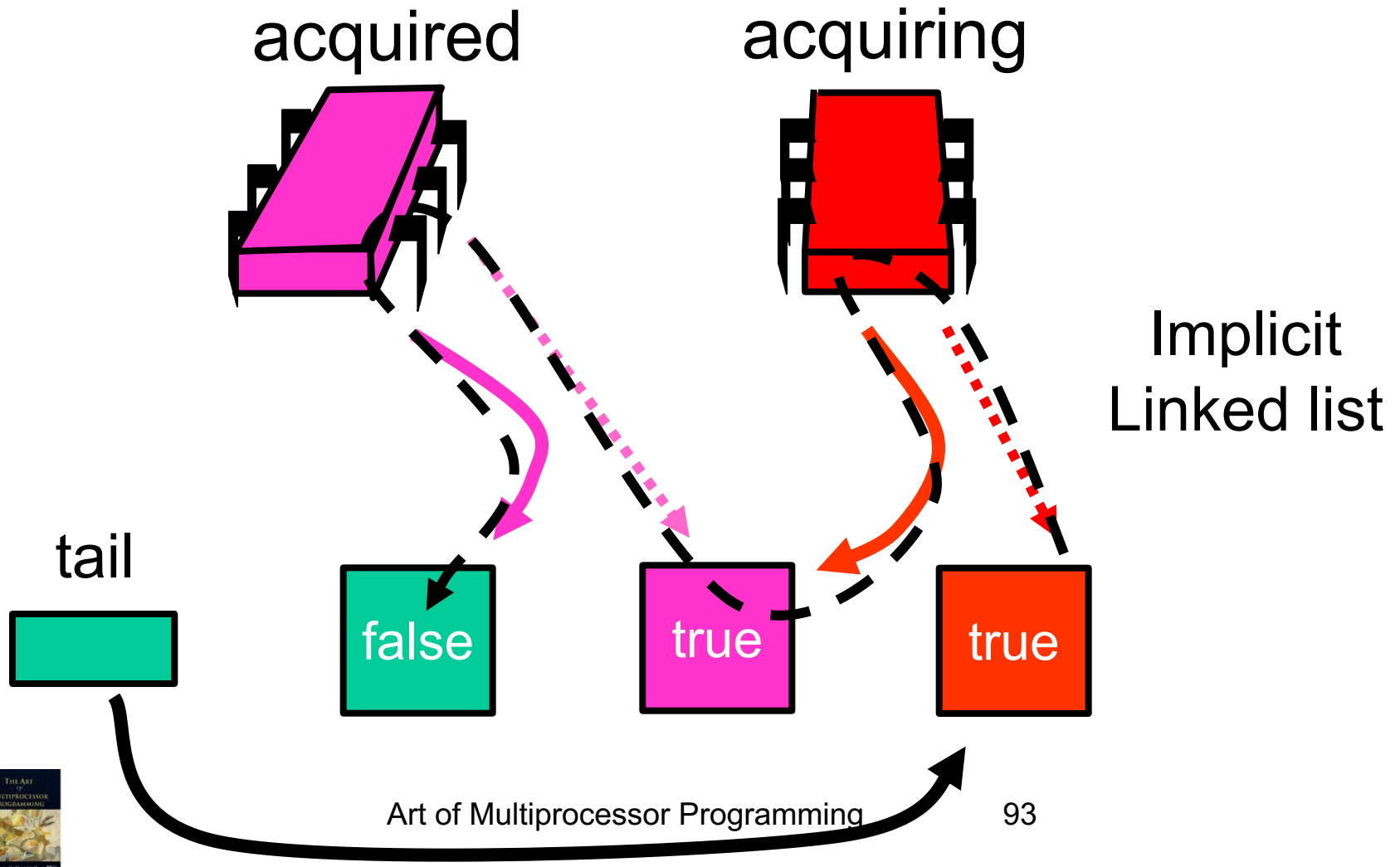
Red Wants the Lock



Red Wants the Lock

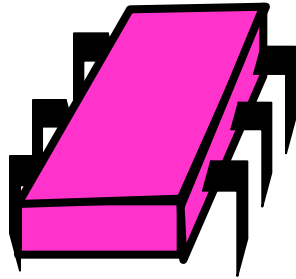


Red Wants the Lock

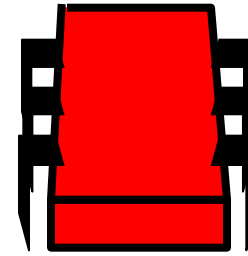


Red Wants the Lock

acquired



acquiring



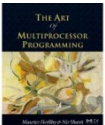
tail



false

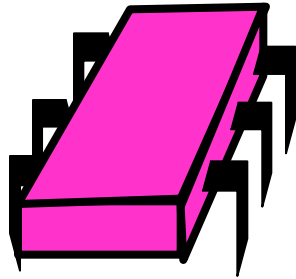
true

true

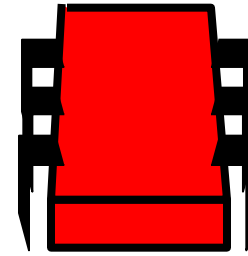


Red Wants the Lock

acquired



acquiring



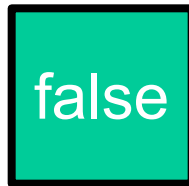
true

Actually, it spins on cached copy

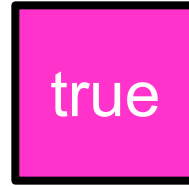
tail



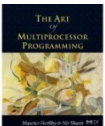
false



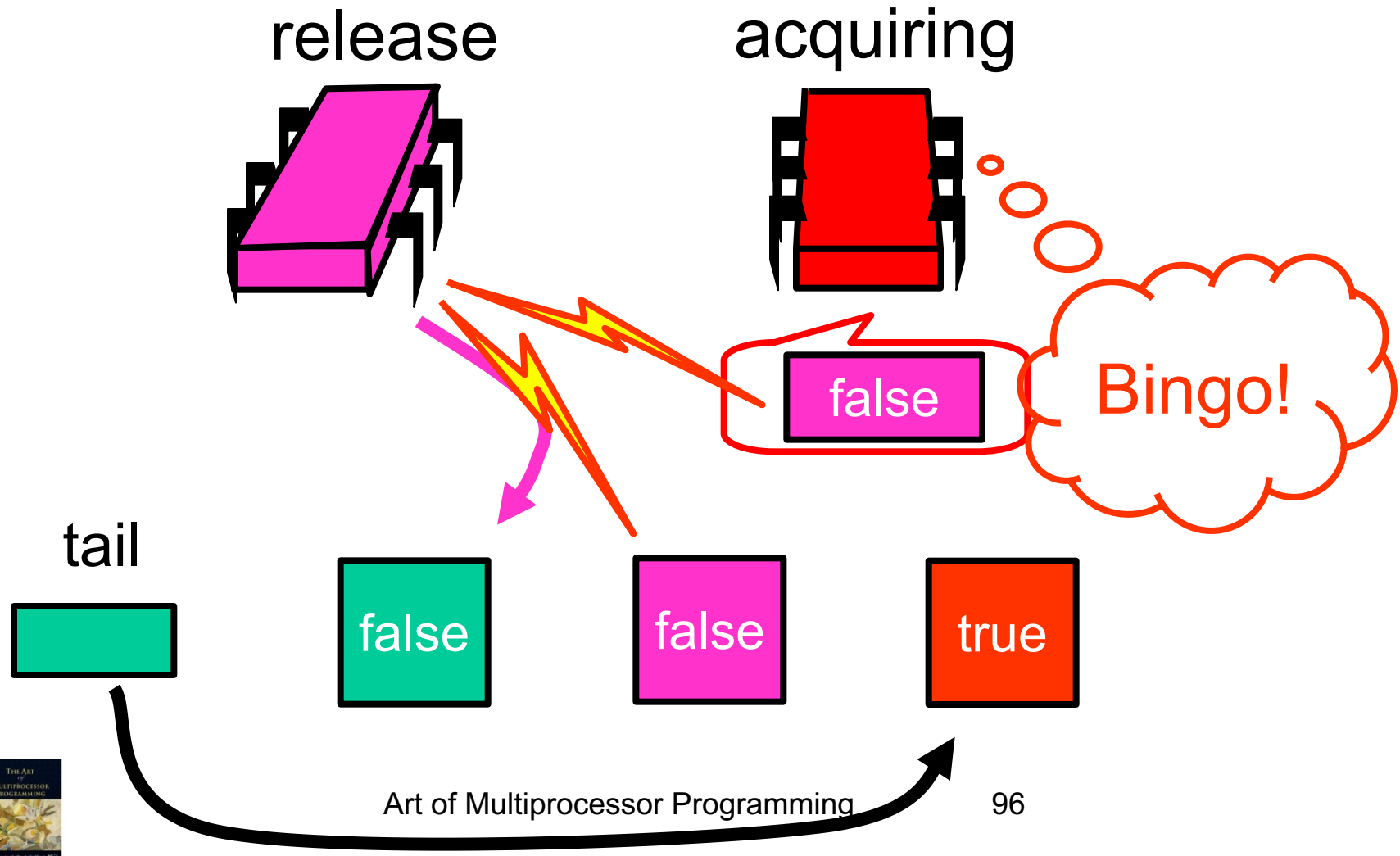
true



true

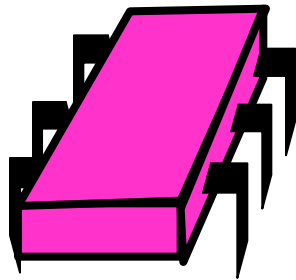


Purple Releases

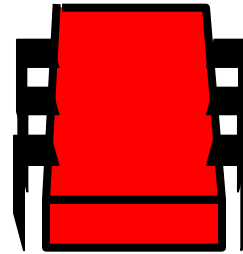


Purple Releases

released



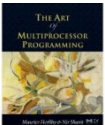
acquired



tail

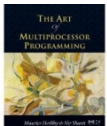


true



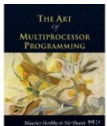
Space Usage

- Let
 - L = number of locks
 - N = number of threads
- ALock
 - $O(LN)$
- CLH lock
 - $O(L+N)$



CLH Queue Lock

```
class Qnode {  
    AtomicBoolean locked =  
        new AtomicBoolean(true);  
}
```



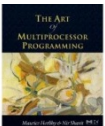
CLH Queue Lock

```
class Qnode {  
    AtomicBoolean locked =  
        new AtomicBoolean(true);  
}
```

Not released yet

CLH Queue Lock

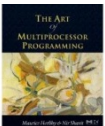
```
class CLHLock implements Lock {  
    AtomicReference<Qnode> tail;  
    ThreadLocal<Qnode> myNode  
        = new Qnode();  
    public void lock() {  
        Qnode pred  
            = tail.getAndSet(myNode);  
        while (pred.locked) {}  
    }  
}
```



CLH Queue Lock

```
class CLHLock implements Lock {  
    AtomicReference<Qnode> tail;  
    ThreadLocal<Qnode> myNode  
        = new Qnode();  
    public void lock() {  
        Qnode pred  
            = tail.getAndSet(myNode);  
        while (pred.locked) {}  
    }  
}
```

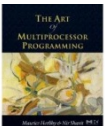
Queue tail



CLH Queue Lock

```
class CLHLock implements Lock {  
    AtomicReference<Qnode> tail;  
    ThreadLocal<Qnode> myNode  
        = new Qnode();  
    public void lock() {  
        Qnode pred  
            = tail.getAndSet(myNode);  
        while (pred.locked) {}  
    }  
}
```

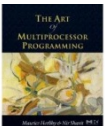
Thread-local Qnode



CLH Queue Lock

```
class CLHLock implements Lock {  
    AtomicReference<Qnode> tail;  
    ThreadLocal<Qnode> myNode  
        = new Qnode();  
    public void lock() {  
        Qnode pred  
            = tail.getAndSet(myNode);  
        while (pred.locked) {}  
    }  
}
```

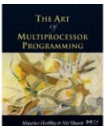
Swap in my node



CLH Queue Lock

```
class CLHLock implements Lock {  
    AtomicReference<Qnode> tail;  
    ThreadLocal<Qnode> myNode  
        = new Qnode();  
    public void lock() {  
        Qnode pred  
            = tail.getAndSet(myNode);  
        while (pred.locked) {}  
    }  
}
```

Spin until predecessor
releases lock



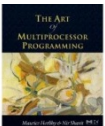
CLH Queue Lock

```
Class CLHLock implements Lock {  
    ...  
    public void unlock() {  
        myNode.locked.set(false);  
    }  
}
```

CLH Queue Lock

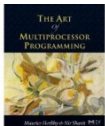
```
Class CLHLock implements Lock {  
    ...  
    public void unlock() {  
        myNode.locked.set(false);  
    }  
}
```

Notify successor



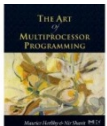
CLH Lock

- Good
 - Lock release affects predecessor only
 - Small, constant-sized space
- Bad
 - Doesn't work for uncached NUMA architectures



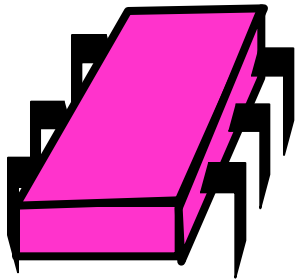
MCS Lock

- FIFO order
- Spin on local memory only
- Small, Constant-size overhead
- Named after Mellor-Crummey and Scott
- https://www.cs.rochester.edu/u/scott/papers/1991_TOCS_synch.pdf

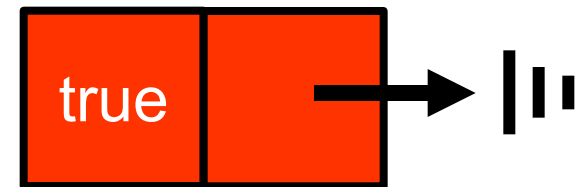
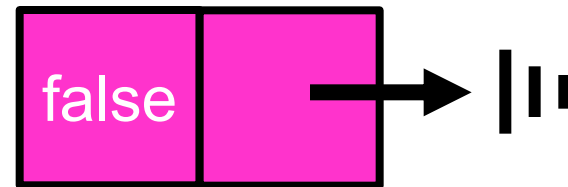
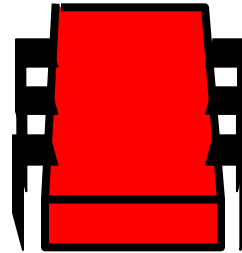


Acquiring

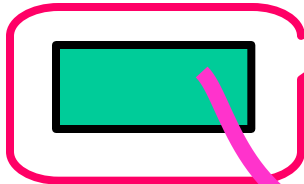
acquired



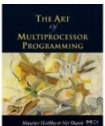
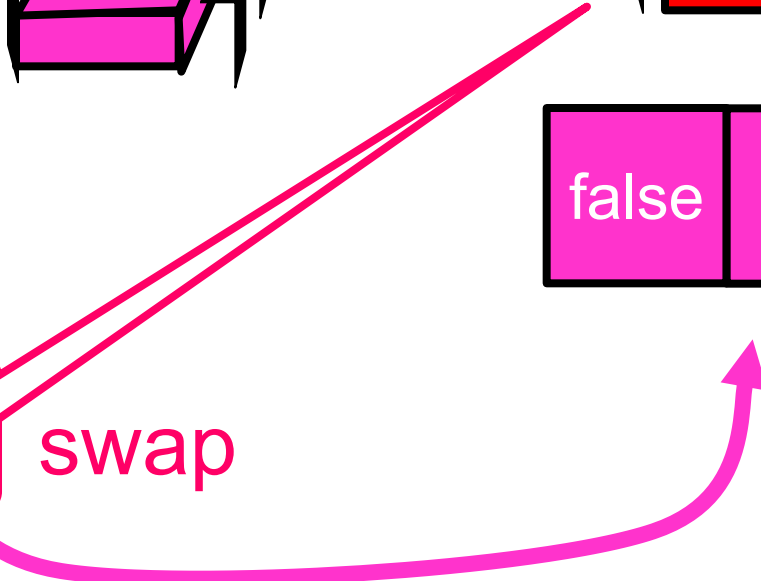
acquiring



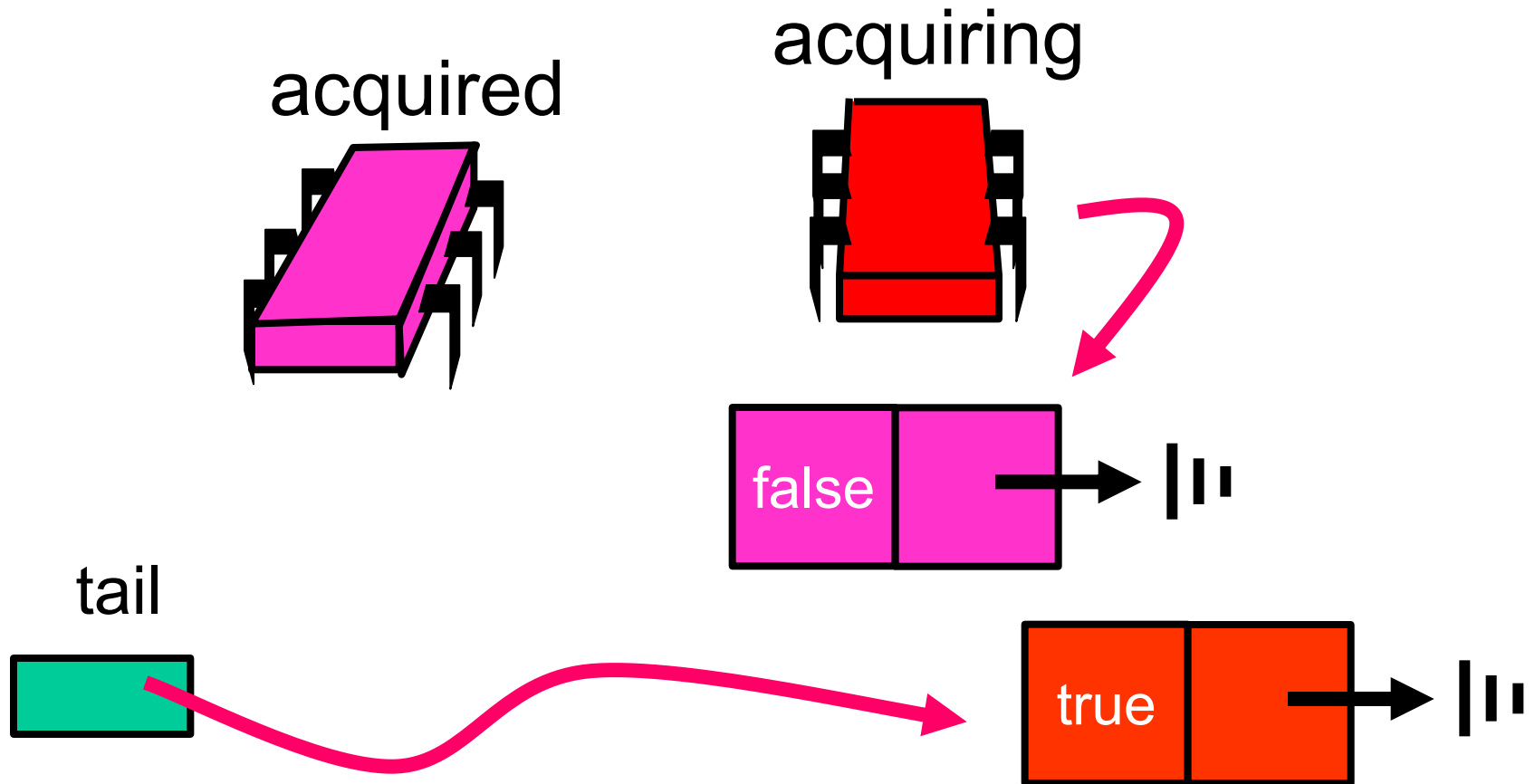
tail



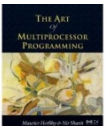
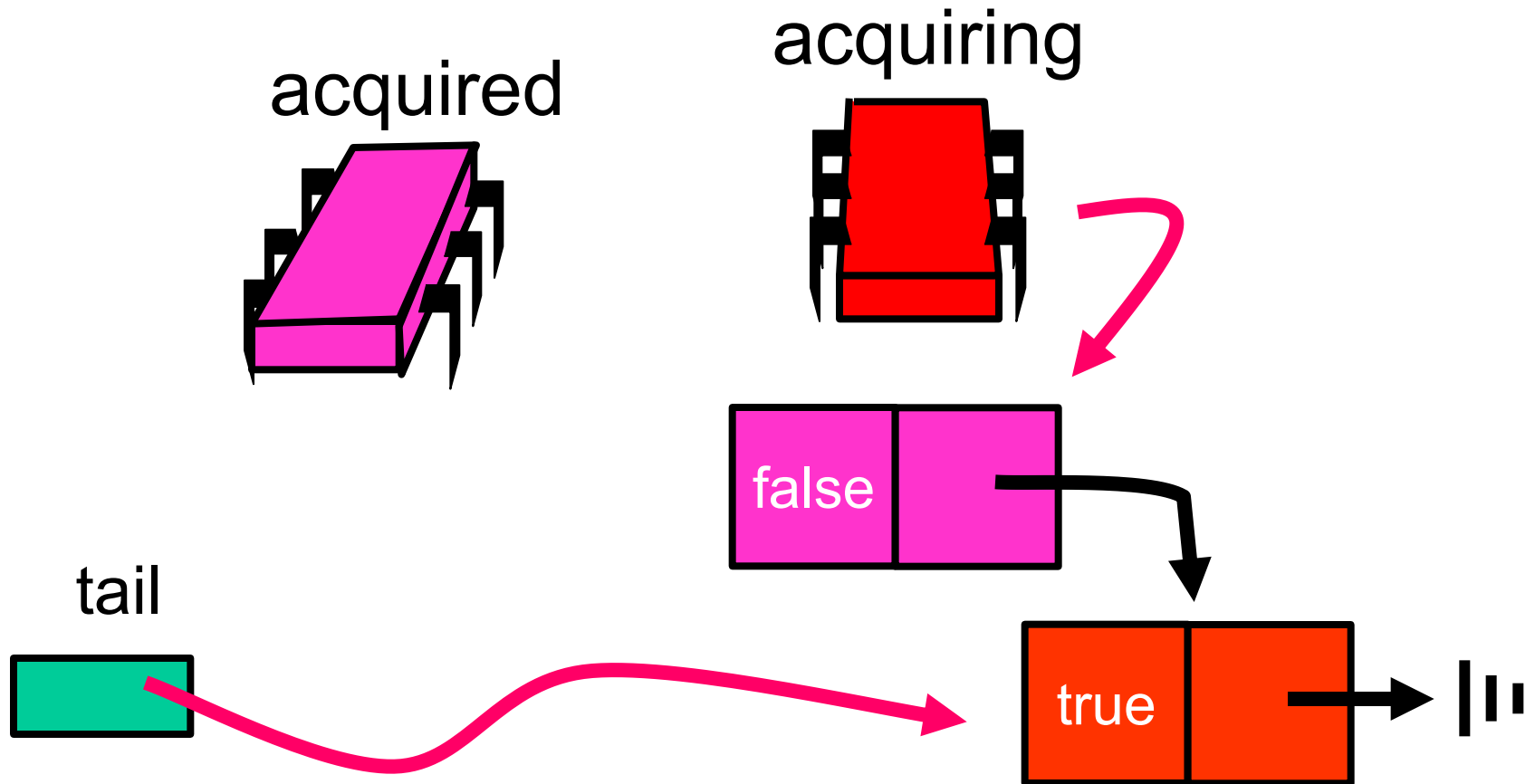
swap



Acquiring



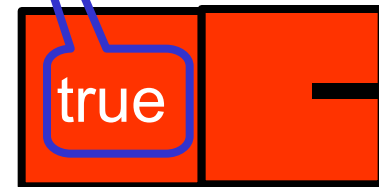
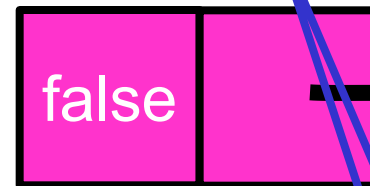
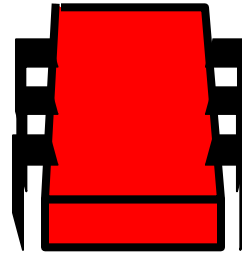
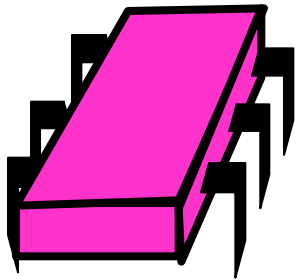
Acquiring



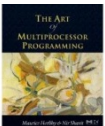
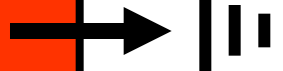
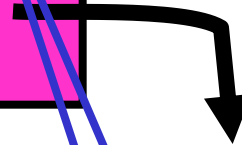
Acquiring

acquiring

acquired



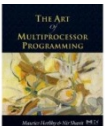
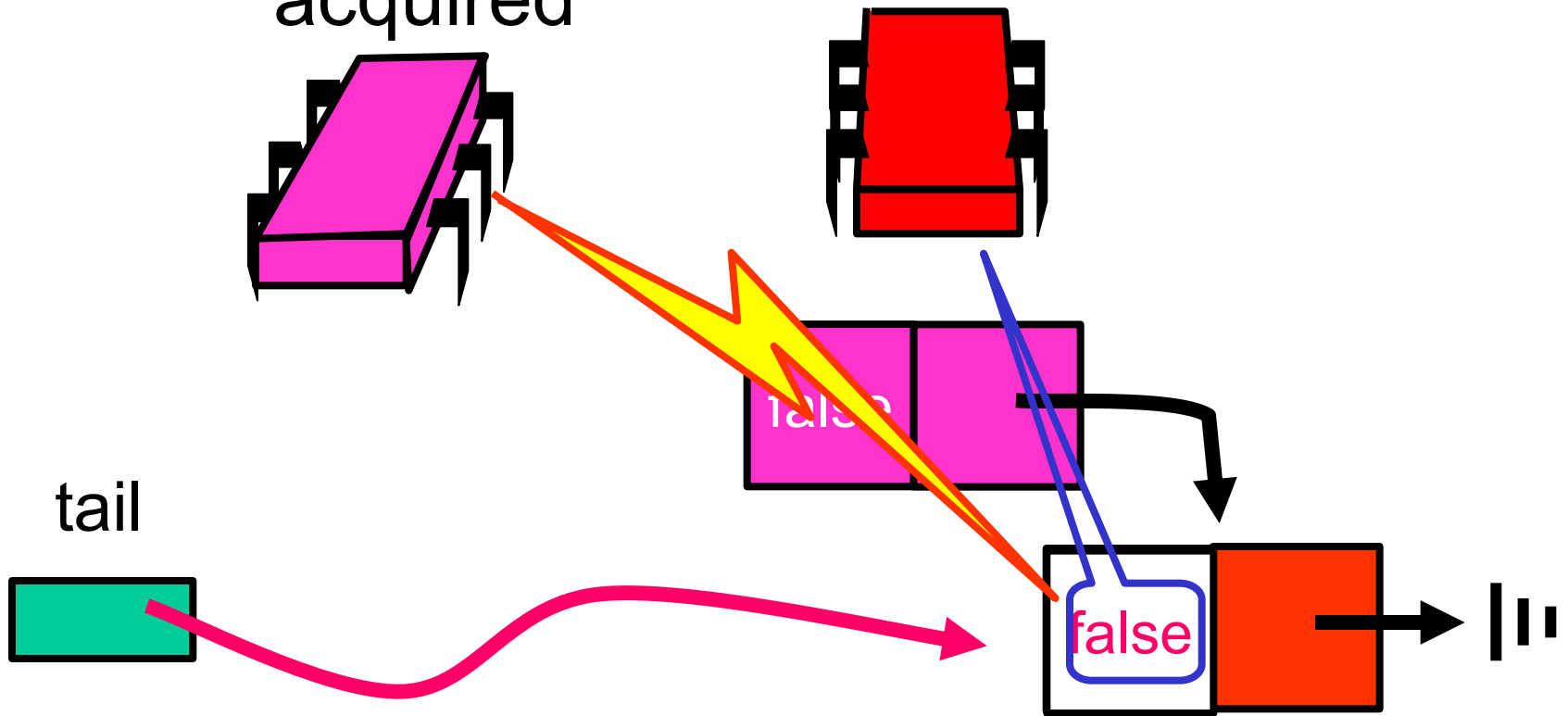
tail



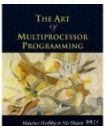
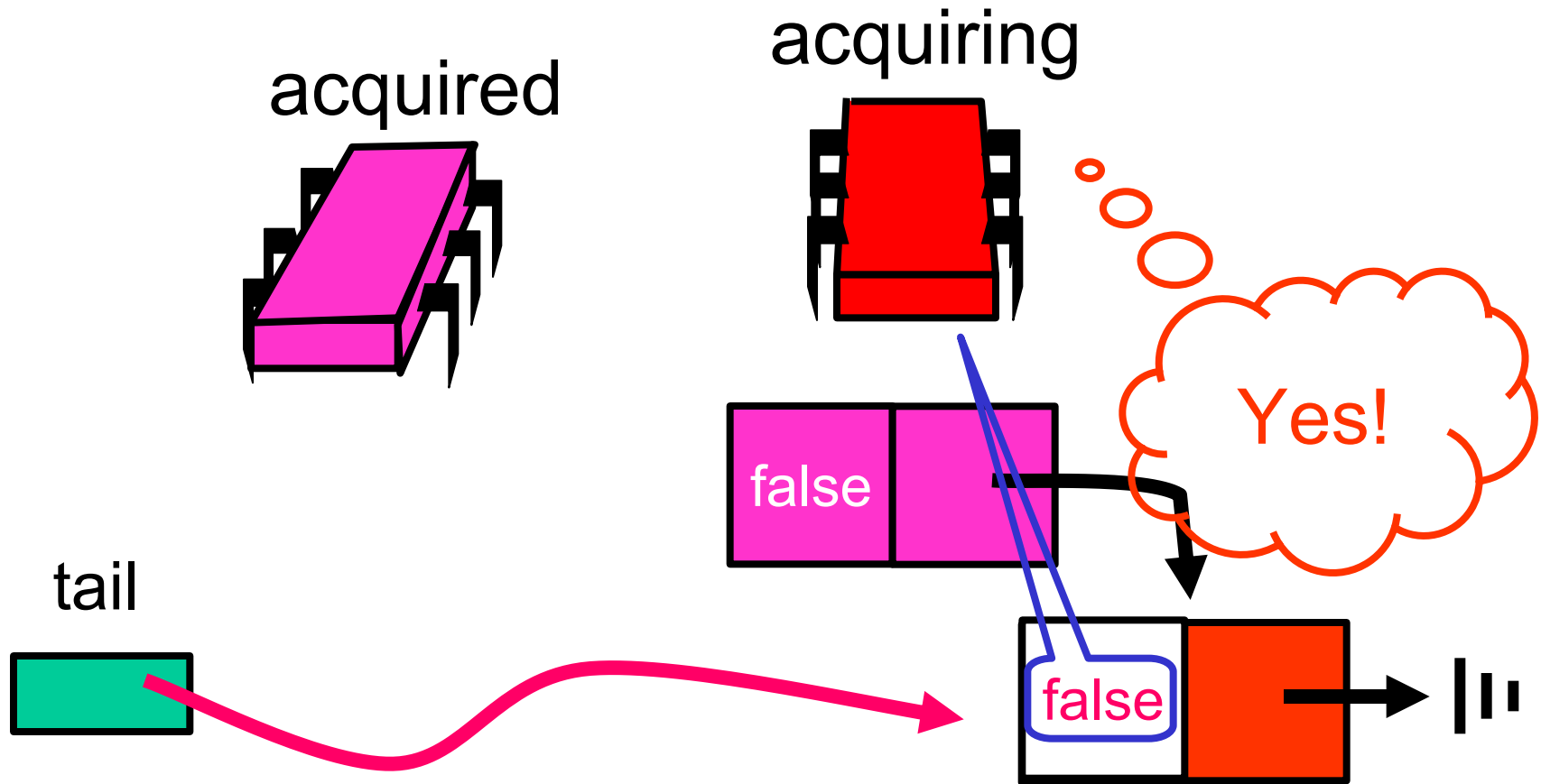
Acquiring

acquiring

acquired

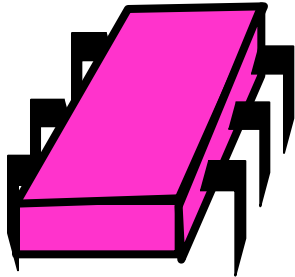


Acquiring

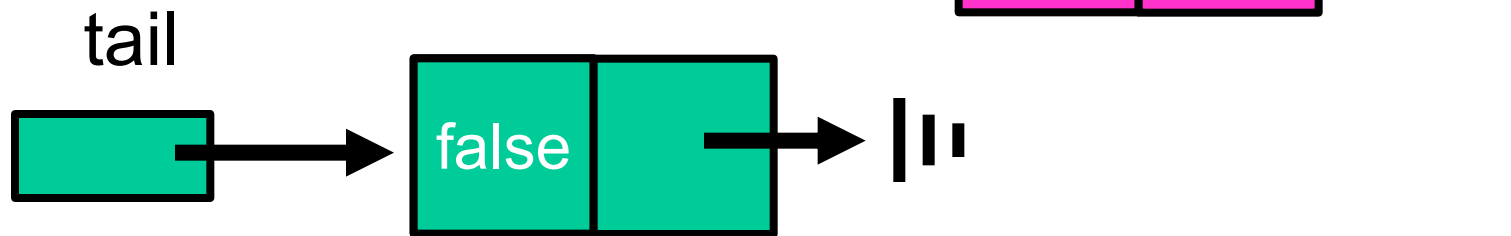


Initially

acquiring

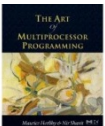


(allocate Qnode)



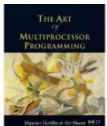
MCS Queue Lock

```
class Qnode {  
    boolean locked = false;  
    qnode  next    = null;  
}
```



MCS Queue Lock

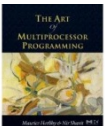
```
class MCSLock implements Lock {  
    AtomicReference tail;  
    public void lock() {  
        Qnode qnode = new Qnode();  
        Qnode pred = tail.getAndSet(qnode);  
        if (pred != null) {  
            qnode.locked = true;  
            pred.next = qnode;  
            while (qnode.locked) {}  
        }  
    }  
}
```



MCS Queue Lock

```
class MCSLock implements Lock {  
    AtomicReference tail;  
    public void lock() {  
        Qnode qnode = new Qnode();  
        Qnode pred = tail.getAndSet(qnode);  
        if (pred != null) {  
            qnode.locked = true;  
            pred.next = qnode;  
            while (qnode.locked) {}  
        }  
    }  
}
```

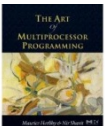
Make a
QNode



MCS Queue Lock

```
class MCSLock implements Lock {  
    AtomicReference tail;  
    public void lock() {  
        Qnode qnode = new Qnode();  
        Qnode pred = tail.getAndSet(qnode);  
        if (pred != null) {  
            qnode.locked = true;  
            pred.next = qnode;  
            while (qnode.locked) {}  
        }  
    }  
}
```

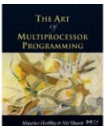
add my Node to
the tail of queue



MCS Queue Lock

```
class MCSLock implements Lock {  
    AtomicReference tail;  
    public void lock() {  
        Qnode qnode = new Qnode();  
        Qnode pred = tail.getAndSet(qnode);  
        if (pred != null) {  
            qnode.locked = true;  
            pred.next = qnode;  
            while (qnode.locked) {}  
        }  
    }  
}
```

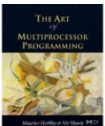
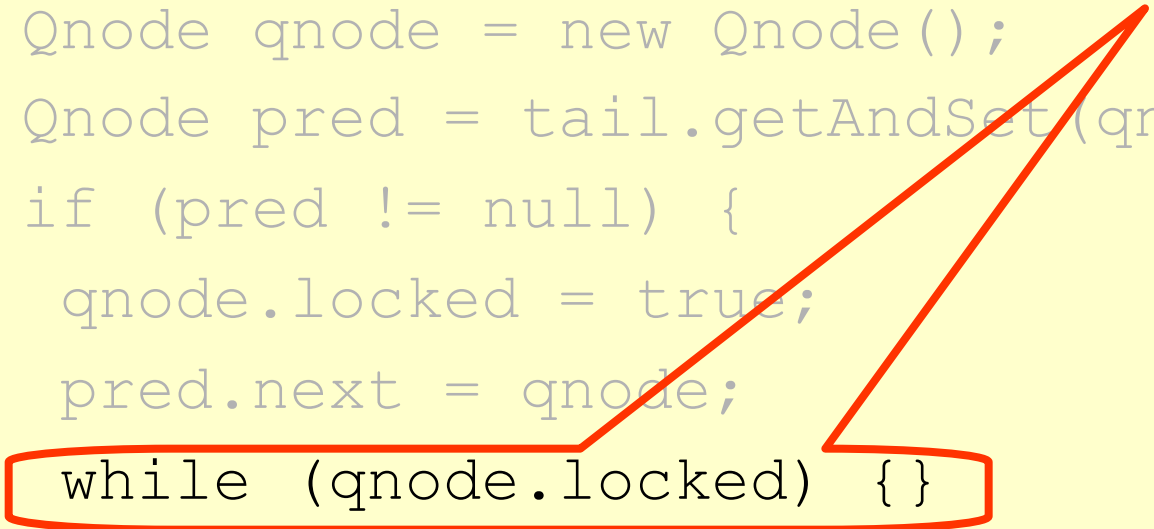
Fix if queue was
non-empty



MCS Queue Lock

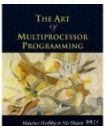
```
class MCSLock implements Lock {  
    AtomicReference tail;  
    public void lock() {  
        Qnode qnode = new Qnode();  
        Qnode pred = tail.getAndSet(qnode);  
        if (pred != null) {  
            qnode.locked = true;  
            pred.next = qnode;  
            while (qnode.locked) {}  
        }  
    }  
}
```

**Wait until
unlocked**



MCS Queue Unlock

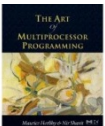
```
class MCSLock implements Lock {  
    AtomicReference tail;  
    public void unlock() {  
        if (qnode.next == null) {  
            if (tail.CAS(qnode, null)  
                return;  
            while (qnode.next == null) {}  
        }  
        qnode.next.locked = false;  
    }  
}
```



MCS Queue Lock

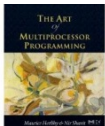
```
class MCSLock implements Lock {  
    AtomicReference tail;  
    public void unlock() {  
        if (qnode.next == null) {  
            if (tail.CAS(qnode, null)  
                return;  
            while (qnode.next == null) {}  
        }  
        qnode.next.locked = false;  
    }  
}
```

**Missing
successor?**



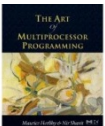
MCS Queue Lock

```
(  
    If really no successor,  
    return  
    public void unlock() {  
        if (qnode.next == null) {  
            if (tail.CAS(qnode, null)  
                return;  
            while (qnode.next == null) {}  
        }  
        qnode.next.locked = false;  
    }  
})
```



MCS Queue Lock

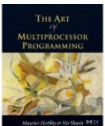
```
(  
    k {  
        Otherwise wait for  
        successor to catch up  
        public void unlock() {  
            if (qnode.next == null) {  
                if (tail.CAS(qnode, null)  
                    return;  
                while (qnode.next == null) {}  
            }  
            qnode.next.locked = false;  
        }  
    }  
}
```



MCS Queue Lock

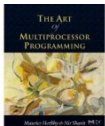
```
class MCSLock implements Lock {  
    AtomicReference<QueueNode> tail = new AtomicReference<>();  
    public void lock() {  
        if (qnode.next == null) {  
            if (tail.CAS(qnode, null))  
                return;  
            while (qnode.next == null) {}  
        }  
        qnode.next.locked = false;  
    }  
}
```

Pass lock to successor



One Lock To Rule Them All?

- TAS, MCS...
- Each better than others in some way
- There is no one solution
- Lock we pick really depends on:
 - the application
 - the hardware
 - which properties are important



Conclusion